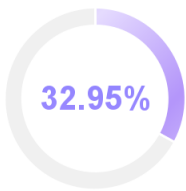


NO. 2ad800b6ee105871 | 2026-05-20 18:36:51

- 题目：基于CNN与YuNet的人脸表情识别系统设计与实现
- 作者：张吉鹏
- 检测所属单位： -

📄 论文字符数：32976 📄 论文页数：58 📊 表格数量：12 🖼️ 图片数量：30

检测结果



32.95% 疑似AIGC生成占比	67.05% 人工撰写占比
-----------------------------	-------------------------

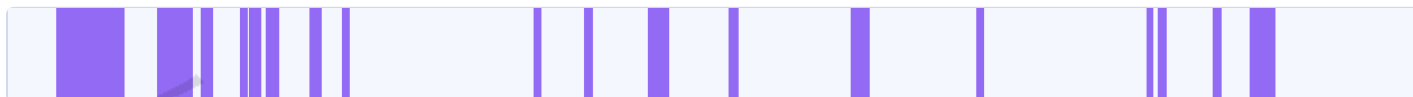
结果分布

序号	章节	疑似AIGC生成文字/章节总字数	疑似AIGC生成章节占比	人工撰写占比
1	摘要	0/815	0.0%	100.0%
2	Abstract	366/378	96.0%	4.0%
3	第1章 绪论	2486/4176	59.0%	41.0%
4	第2章 系统需求分析与架构设计	1142/4917	23.0%	77.0%
5	第3章 表情识别模型构建与训练	1341/5922	22.0%	78.0%
6	第4章 模型训练优化	1073/3687	29.0%	71.0%
7	第5章 系统实现与运行测试	1057/5730	18.0%	82.0%

*注:格式规范的情况下可准确识别章节, 若论文中无章节, 可能会识别有误。

片段分布

■ 疑似AIGC生成 ■ 人工撰写



文字标注

■ 疑似AIGC生成 ■ 人工撰写

基于CNN与YuNet的人脸表情识别系统设计与实现



天津中德应用技术大学
Tianjin Sino-German University of Applied Sciences

本科生毕业设计

基于CNN与YuNet的人脸表情识别系统设计与实现

Design and Implementation of Facial Expression Recognition System Based on CNN and YuNet

学 院 智能制造学院

专 业 自动化

班 级 22自动化1班

学 号 2240401013

姓 名 张吉鹏

指导教师 康莉莉

职 称 副教授

完成时间 2026年06月01日

天津中德应用技术大学

本科生毕业设计

基于CNN与YuNet的人脸表情识别系统设计与实现

Design and Implementation of Facial Expression Recognition System Based on CNN and YuNet

学 院 智能制造学院

专 业 自动化

班 级 22自动化1班

学 号 2240401013

姓 名 张吉鹏

指导教师 康莉莉

职 称 副教授

完成时间 2026年06月01日

天津中德应用技术大学

本科生毕业设计（论文）的声明

本人郑重声明：所呈交的毕业设计（论文），是本人在指导教师指导下，进行研究工作所取得的成果。除文中已经注明引用的内容外，本毕业设计（论文）的研究成果不包含任何他人创作的、已公开发表或没有公开发表的作品内容。对本设计（论文）所涉及的研究工作做出贡献的其他个人和集体，均已在文中以明确方式标明。本毕业设计（论文）原创性声明的法律 responsibility 由本人承担。

毕业设计（论文）作者签名：

2026年06月01日

本人声明：该毕业设计（论文）是本人指导学生完成的研究成果，已经审阅过设计（论文）的全部内容，并能够保证题目、关键词、摘要部分中英文内容的一致性和准确性。

毕业设计（论文）指导教师签名：

2026年06月01日

摘 要

本文围绕树莓派端实时人脸表情识别任务，完成了一套基于CNN与YuNet的人脸表情识别系统。系统以摄像头实时画面为输入，先利用YuNet检测人脸位置和关键点，再将裁剪后的人脸区域输入MobileNetV3表情分类模型，最终在显示界面中输出人脸框、表情类别、置信度和运行状态信息。与只进行离线图像分类的实验不同，本文更关注模型训练、格式转换和端侧运行之间的衔接过程。

在表情分类模型训练方面，本文以FER2013格式数据为基础，对样本进行了扩充、合并、清洗和重新划分，构建训练集、验证集、测试集和无标签集。模型采用MobileNetV3-large作为骨干网络，并将最终分类层调整为七类表情输出，用于识别愤怒、厌恶、恐惧、高兴、悲伤、惊讶和中性。训练过程中结合数据增强、类别均衡、标签平滑、学习率调度和伪标签辅助训练等方法，以提高模型在类别不均衡条件下的训练稳定性。

在系统实现方面，本文完成了PyTorch模型训练、ONNX模型导出、TensorFlow SavedModel转换和TensorFlow Lite模型生成，并将FP16量化后的TFLite模型部署到Raspberry Pi 5平台。树莓派端程序集成了摄像头读取、YuNet人脸检测、人脸区域裁剪、TFLite表情分类、结果绘制和图像保存等功能。为了降低端侧运行负载，程序采用缩小

检测输入尺寸、间隔执行检测、限制单帧分类人脸数量和异步读取摄像头画面等方式提高运行连续性。

实验结果表明，最终Stage 3表情分类模型在测试集上取得90.77%的准确率和87.43%的宏平均F1值。树莓派端运行测试表明，系统能够在室内近距离场景下完成实时人脸检测与表情识别，运行画面较为连续，检测框、关键点、类别和置信度能够正常显示，连续运行过程中未观察到程序崩溃、摄像头中断或界面卡死等异常情况。结果说明，本文系统实现了从模型训练到端侧部署运行的完整流程，具备一定的实时性、稳定性和工程应用价值。

关键词：人脸表情识别；卷积神经网络；YuNet；MobileNetV3；树莓派；端侧部署

ABSTRACT

This paper designs and implements a facial expression recognition system based on CNN and YuNet for real-time edge deployment on a Raspberry Pi. The system takes camera frames as input, detects facial regions and landmarks using YuNet, and then feeds the cropped face region into a MobileNetV3-based expression classification model. The recognition result, including the face bounding box, expression category, confidence score, and runtime information, is displayed on the screen. Compared with an offline image classification experiment, this work focuses more on the complete process from model training and model conversion to edge-side real-time operation.

For expression classification, datasets in the FER2013 format are used as the basis. The samples are expanded, merged, cleaned, and re-split into training, validation, test, and unlabeled sets. MobileNetV3-large is adopted as the backbone network, and its final classification layer is modified to output seven expression categories, including anger, disgust, fear, happiness, sadness, surprise, and neutrality. During training, data augmentation, class balancing, label smoothing, learning rate scheduling, and pseudo-label-assisted training are used to improve training stability under imbalanced class distribution.

For system implementation, the trained PyTorch model is exported to ONNX, converted to TensorFlow SavedModel, and finally converted to TensorFlow Lite. The FP16-quantized TFLite model is deployed on the Raspberry Pi 5 platform. The edge-side program integrates camera reading, YuNet-based face detection, face cropping, TFLite expression inference, result visualization, and image saving. To reduce the computational load on the edge device, the program uses a smaller detection input size, interval-based face detection, limited per-frame face classification, and asynchronous camera reading.

Experimental results show that the final Stage 3 expression classification model achieves an accuracy of 90.77% and a macro-average F1 score of 87.43% on the test set. Runtime tests on the Raspberry Pi show that the system can perform real-time face detection and facial expression recognition in indoor close-range scenarios. The video display remains relatively smooth, and the bounding box, landmarks, expression category, and confidence score can be updated normally. No program crash, camera interruption, or interface freeze is observed during continuous operation.

These results indicate that the system completes the full workflow from model training to edge deployment and has practical real-time performance, stability, and engineering value.

Key Words: facial expression recognition; convolutional neural network; YuNet; MobileNetV3; Raspberry Pi; edge deployment

目 录

第1章 绪论1

1.1 研究背景与意义1

1.2 国内外研究现状2

1.3 本文主要研究内容3

1.4 本文结构安排4

第2章 系统需求分析与架构设计5

2.1 系统需求分析5

2.1.1 应用场景与任务目标5

2.1.2 功能需求分析6

2.1.3 性能需求分析6

2.2 系统总体架构设计7

2.2.1 系统总体架构概述7

2.2.2 系统识别流程设计9

2.2.3 系统功能模块划分10

2.3 关键技术与方法选型11

2.3.1 人脸检测模型选型11

2.3.2 表情分类模型选型12

2.3.3 训练优化方法选型13

2.4 本章小结14

第3章 表情识别模型构建与训练15

3.1 数据集构建与预处理15

3.1.1 数据集构建与扩展15

3.1.2 CSV数据合并与数据集划分16

3.1.3 数据预处理与增强策略18

3.1.4 类别不平衡处理18

3.2 模型架构设计19

3.2.1 MobileNetV3骨干网络选型19

3.2.2 分类层调整与输出设计19

3.2.3 模型评价指标19

3.3	训练策略设计	20
3.3.1	基础监督训练方案	20
3.3.2	伪标签辅助训练方案	20
3.3.3	多阶段训练流程设计	21
3.3.4	训练参数设置	22
3.4	本章小结	22
第4章	模型训练优化	23
4.1	训练优化思路与总体流程	23
4.2	多阶段训练结果对比	23
4.3	最终模型总体评价	24
4.4	混淆矩阵与类别识别分析	25
4.5	本章小结	27
第5章	系统实现与运行测试	28
5.1	系统实现与运行流程	28
5.2	软件环境与硬件平台配置	28
5.2.1	硬件平台配置	28
5.2.2	软件环境与工具	29
5.3	模型导出与端侧部署	30
5.3.1	模型导出与格式转换	30
5.3.2	模型转换一致性验证	31
5.4	树莓派端推理程序设计	31
5.4.1	推理程序总体结构	31
5.4.2	树莓派端运行参数	32
5.5	系统运行测试与效果分析	33
5.5.1	功能测试	33
5.5.2	实时性观察	33
5.5.3	运行效果展示	34
5.5.4	稳定性测试	36
5.6	本章小结	36
第6章	结论与展望	37
6.1	结论	37
6.2	展望	38
	参考文献	39
	致谢	41

第1章 绪论

1.1 研究背景与意义

随着人工智能、计算机视觉和嵌入式计算技术的发展，智能系统的功能已经从简单的信息处理逐渐扩展到对用户状态的感知与反馈。人脸表情是人类表达情绪的重要方式之一，具有直观性强、采集方便和非接触等特点，因此人脸表情识别逐渐成为情感计算和人机交互领域中的重要研究方向[1-3]。近年来，人脸表情识别已被应用于智能交互、课堂状态分析、驾驶员状态监测、心理健康辅助和服务机器人等场景，其研究价值不仅体现在识别算法本身，也体现在实际系统的部署和运行能力上。

人脸表情识别的基本任务是从图像或视频中获取人脸区域，并对其情绪类别进行判断。常见表情类别通常包括愤怒、厌恶、恐惧、高兴、中性、悲伤和惊讶等。FER2013等公开数据集为表情识别模型训练和方法对比提供了基础数据来源[4]，使不同模型能够在较统一的数据标准下进行测试与评价。随着深度学习的发展，卷积神经网络逐渐取代传统人工特征方法，成为人脸表情识别中的主流技术路线。相比传统纹理特征和浅层分类器，卷积神经网络能够从图像中自动提取多层次特征，在复杂表情图像识别中具有更强的表达能力。

但是，人脸表情识别系统在实际应用中并不只需要关注离线测试准确率。对于树莓派、移动终端和边缘计算设备等资源受限平台而言，模型的参数量、计算复杂度、推理速度和运行稳定性同样重要[12-15]。较复杂的深度模型虽然可能在数据集上取得较高准确率，但在端侧平台上运行时容易出现推理延迟高、画面卡顿、内存占用大和连续运行不稳定等问题。因此，如何在保证基本识别效果的前提下，降低模型计算量并完成端侧部署，是当前人脸表情识别系统设计中需要重点解决的问题[1][12][13]。

在轻量化表情识别研究中，研究者通常从网络结构优化、注意力机制、特征融合和模型压缩等角度提升模型效率。例如，基于轻量化ResNet、MobileNet和Transformer改进的表情识别方法，均尝试在参数量、计算量和识别准确率之间取得平衡[5-8]。其中，MobileNetV3由于具有较好的移动端适应性，适合作为嵌入式视觉任务中的轻量化骨干网络[9]。在人脸检测环节，YuNet作为面向边缘设备设计的轻量级人脸检测模型，能够在较低计算开销下完成人脸定位和关键点检测，为端侧实时表情识别提供了可行的前端检测方案[10]。

基于上述背景，本毕业设计围绕端侧人脸表情识别系统的设计与实现展开研究。系统采用“轻量化人脸检测 + 轻量化表情分类 + 树莓派端部署”的整体方案，前端利用YuNet完成人脸检测与关键点定位，后端基于MobileNetV3构建七类表情分类模型，并将训练后的模型部署到树莓派平台，实现从视频采集、人脸检测、表情分类到结果显示的完整流程。相比单纯进行离线模型训练，本设计更强调模型训练、模型转换、端侧部署和运行测试之间的闭环实现，具有较强的工程实践意义。

1.2 国内外研究现状

人脸表情识别技术经历了由传统人工特征方法向深度学习方法发展的过程。早期方法通常先进行人脸检测和预处理，再提取局部纹理、几何结构或统计特征，最后结合分类器完成表情判断。这类方法结构较清晰，计算量相对可控，但对光照变化、姿态偏转、遮挡和个体差异较敏感，在复杂自然场景下识别稳定性有限。近年来的综述研究表明，深度学习已经成为人脸表情识别领域的主流方向，研究重点也逐渐从单一模型准确率扩展到数据质量、模型

泛化能力、轻量化设计和实际部署等方面[1-3]。

在深度学习方法中，卷积神经网络能够直接从表情图像中学习特征，减少人工设计特征对经验的依赖。FER2013数据集的提出推动了深度表情识别方法的发展，使模型可以围绕七类基本表情进行训练和测试[4]。此后，研究者通过改进卷积网络结构、引入注意力机制、融合多尺度特征等方式提高表情识别效果。赵晓等提出基于注意力机制的ResNet轻量网络，通过分组卷积、倒残差结构和多尺度注意力模块减少模型参数量，并在FER2013和CK+数据集上进行了验证[5]。这类研究说明，轻量化设计不仅可以降低模型复杂度，也可以通过合理的特征增强方法保持模型的识别能力。

MobileNet系列网络是轻量化卷积神经网络中的代表结构之一。左义海等提出NCA-MobileNet方法，通过改进MobileNetV3结构并引入注意力机制，实现了面向人脸表情识别任务的轻量化优化[6]。李艳秋等将轻量型Swin Transformer与多尺度特征融合模块相结合，用于改善模型在表情细节捕捉和实时性方面的表现[7]。Jiang等则围绕改进MobileNetV3的人脸表情识别算法展开研究，说明MobileNetV3在表情识别任务中仍具有进一步优化和应用价值[8]。MobileNetV3原始模型通过硬件感知网络搜索和结构优化，在移动端视觉任务中实现了较好的精度与效率平衡[9]。因此，本文选用MobileNetV3作为表情分类模型的骨干网络，能够较好地契合树莓派端部署对模型规模和推理效率的要求。

在人脸检测方面，检测结果的准确性会直接影响后续表情分类输入质量。传统或较复杂的人脸检测模型虽然能够取得较高检测精度，但在端侧平台上运行时可能带来较高计算开销。YuNet是一种面向边缘设备设计的轻量级人脸检测模型，具有模型小、速度快和便于集成等特点，适合作为实时表情识别系统的前端检测模块[10]。此外，也有研究通过改进S3FD等检测网络提高复杂场景下的人脸检测效果[11]。对于本文所设计的系统而言，检测模块不只需要找到人脸，还需要保证在视频流中连续运行，因此轻量化检测方案更符合树莓派端应用需求。

随着边缘计算和嵌入式AI的发展，人脸表情识别系统的研究逐渐从PC端离线测试转向端侧实时部署。Mohammadi等针对边缘设备上的人脸表情识别进行了实验，比较了CPU、GPU、VPU和TPU等不同硬件平台下的运行表现，说明FER模型的实际应用效果与硬件平台、模型结构和推理优化方式密切相关[12]。Liu等对边缘设备上的高效神经网络进行了综述，指出量化、剪枝和网络结构设计是提高端侧模型运行效率的重要方向[13]。Weng等对神经网络量化方法进行了总结，认为量化可以通过降低数值精度来减少模型复杂度和推理开销[14]。Wei等进一步综述了神经网络量化技术的发展，说明模型压缩和低精度推理仍是端侧部署中的重要研究内容[15]。

综合国内外研究可以看出，人脸表情识别已经形成较为完整的发展路径：传统方法侧重人工特征提取，深度学习方法提升了模型的特征表达能力，轻量化网络和边缘部署技术则推动表情识别系统从离线实验走向实际应用。但在树莓派等资源受限平台上同时完成人脸检测、表情分类、结果显示和稳定运行，仍需要对模型选型、训练过程、模型转换和端侧推理流程进行综合设计。基于此，本文将研究重点放在YuNet人脸检测、MobileNetV3表情分类和树莓派端部署测试上，力求完成一套能够实际运行的端侧人脸表情识别系统。

1.3 本文主要研究内容

围绕端侧人脸表情识别系统的设计与实现，本文主要开展以下几方面工作。

第一，完成系统需求分析与总体架构设计。结合树莓派端实时视觉识别场景，明确系统需要完成摄像头视频采集、人脸检测、人脸区域裁剪、表情分类、结果显示和图像保存等功能。在此基础上，设计系统整体流程，将系统

划分为图像采集模块、人脸检测模块、图像预处理模块、表情分类模块、结果显示模块和数据保存模块，为后续模型训练和系统实现提供结构基础。

第二，完成基于MobileNetV3的表情分类模型构建。以FER2013格式数据为训练和测试基础，对图像数据进行尺寸调整、归一化和数据增强处理。模型以MobileNetV3为骨干网络，并将最终分类层调整为七类表情输出。训练过程中结合类别均衡、标签平滑、学习率调度和早停机制等方法提升训练稳定性。对于伪标签相关方法，本文将其作为训练优化策略使用，不将其作为核心创新点进行强调。

第三，完成YuNet人脸检测与表情分类模型的系统集成。系统前端采用YuNet对摄像头输入图像进行人脸检测，并根据检测框裁剪人脸区域；后端将裁剪后的人脸图像输入MobileNetV3表情分类模型，得到对应表情类别和置信度结果。通过检测与分类模块的组合，实现从视频帧输入到表情识别结果输出的完整处理流程。

第四，完成模型转换与树莓派端部署。训练完成后，将PyTorch模型转换为适合端侧运行的格式，并在树莓派平台上完成推理程序设计。针对端侧平台算力有限的问题，系统在实际部署时采用降低检测输入分辨率、间隔执行人脸检测、限制单次分类人脸数量和异步读取视频帧等方式减少运行负载，提高系统实时性和连续运行能力。

第五，完成模型评价与系统运行测试。通过测试集准确率、宏平均F1值和混淆矩阵等指标评价表情分类模型性能；通过功能测试、实时性测试和稳定性测试评价树莓派端系统运行效果。最终从模型识别效果和端侧运行效果两个角度，验证本文系统是否完成从训练到部署的完整闭环。

1.4 本文结构安排

本文共分为六章，各章内容安排如下。

第1章为绪论。主要介绍人脸表情识别的研究背景与意义，梳理传统方法、深度学习方法、轻量化网络和端侧部署相关研究现状，并说明本文的主要研究内容和结构安排。

第2章为系统需求分析与架构设计。主要分析端侧人脸表情识别系统的应用场景、功能需求和性能需求，设计系统总体架构，并对图像采集、人脸检测、表情分类和结果输出等模块进行划分。

第3章为表情识别模型构建与训练。主要介绍数据集构建与预处理方法，说明MobileNetV3表情分类网络的结构调整过程，并对数据增强、类别均衡和训练配置等内容进行说明。

第4章为模型训练优化与实验结果分析。主要围绕模型训练过程展开，分析基础监督训练、多阶段训练和伪标签辅助优化对模型效果的影响，并通过准确率、宏平均F1值和混淆矩阵等指标评价最终模型性能。

第5章为系统实现、模型部署与运行测试。主要介绍模型导出与转换流程、树莓派端推理程序设计、摄像头实时识别流程集成，以及系统功能、实时性和稳定性测试结果，重点验证系统在端侧平台上的实际运行效果。

第6章为结论与展望。主要总结本文完成的工作，分析系统当前存在的不足，并从模型优化、多场景适应性和端侧部署性能等方面提出后续改进方向。

第2章 系统需求分析与架构设计

2.1 系统需求分析

本课题设计的对象是一套运行在树莓派端的人脸表情识别系统。系统的基本任务是：通过摄像头获取人脸画面，识别人脸位置，并判断当前人脸所属的表情类别。与只在PC端完成图像分类实验不同，本设计最后需要在实际设备上运行，因此第2章主要从系统使用场景、功能目标和总体架构角度进行分析，具体的数据集处理、模型训练过

程和运行测试结果分别放在第3章、第4章和第5章中展开。

本文系统主要识别七类基本表情，包括愤怒、厌恶、恐惧、高兴、中性、悲伤和惊讶。系统运行时，摄像头采集实时图像，树莓派端程序先完成人脸检测，再将人脸区域送入表情分类模型，最后在画面中显示检测框、表情类别和置信度。结合后续实验目标，表情分类模型在测试集上的准确率应达到90%左右，同时端侧程序应能够保持连续运行，避免出现画面明显卡顿、摄像头中断或程序异常退出等情况。

本设计的应用场景定位为室内近距离人机交互场景，例如课堂状态观察、嵌入式视觉演示、智能终端交互实验等。这类场景下，摄像头通常固定在桌面或设备附近，用户与摄像头距离较近，环境变化相对有限。因此，本文不把远距离监控、复杂多人遮挡和强光照变化作为主要目标，而是重点完成单人或少量人脸情况下的实时检测与表情识别流程。

从实际实现角度看，系统包括两个工作环境：PC端和树莓派端。PC端主要用于数据处理、模型训练和模型转换，硬件环境包含RTX3060 GPU；树莓派5作为最终部署平台，负责摄像头图像采集、人脸检测、表情分类和结果显示。这样的划分能够利用PC端较强的训练能力，同时验证模型在端侧设备上的实际运行效果。

2.1.1 应用场景与任务目标

本文系统面向的是轻量化、可运行、可展示的人脸表情识别原型，而不是单纯追求公开数据集最高精度的算法模型。毕业设计需要完成从模型训练到端侧部署的完整流程，因此系统目标主要包括以下几个方面。

第一，完成实时人脸图像输入。系统应能够调用摄像头获取视频画面，并将图像帧传递给后续识别程序。摄像头画面是系统的输入来源，采集过程是否稳定会直接影响后续检测和分类效果。

第二，完成人脸区域定位。系统需要从视频帧中找到人脸位置，并输出可用于裁剪的人脸框。由于表情分类模型只处理人脸区域，人脸检测框的准确性会影响分类输入质量。如果检测框偏移过大，分类模型可能接收到不完整的人脸图像，从而影响识别结果。

第三，完成七类表情分类。系统需要对检测到的人脸区域进行预处理，并输入表情分类模型，输出七类表情的预测结果。为便于观察模型判断情况，系统不仅显示类别名称，也显示对应置信度。

第四，完成端侧显示与运行验证。系统应在树莓派端显示实时画面，并叠加人脸框、关键点、表情类别和运行状态信息。通过实际运行画面，可以判断系统是否真正完成了从图像输入到结果输出的闭环。

从以上目标可以看出，本设计的重点不是单独改进某一个网络结构，而是把人脸检测模型、表情分类模型和树莓派端程序结合起来，使系统能够在实际设备上连续运行。

2.1.2 功能需求分析

按照任务目标，系统主要包含图像采集、人脸检测、人脸预处理、表情分类、结果显示、结果保存这六个功能。

图像采集功能可以得到实时的视频帧。本文用摄像头做为输入设备，树莓派端程序读取摄像头画面之后，把当前帧送入后续处理流程。由于实时画面不能长时间停留，所以程序应该尽可能减少图像读取过程中对主流程的阻碍。

人脸检测功能就是找到图像里的人脸位置。本文使用YuNet做前端的检测模型，检测结果包含人脸框、关键点信息。人脸框用来截取表情分类所需要的面部区域，关键点可以辅助观察检测位置是否合理。

人脸预处理功能是对检测出的人脸区域进行转换，使其可以被分类模型所接受。该过程就是对边界做修改，区

域裁剪，尺寸缩放，颜色通道转换和归一化处理。预处理过程要和训练阶段保持一致，否则模型在端侧推理的时候就会出现输入分布偏差。

表情分类功能是用来判定人脸图像所对应的表情类别的。本文以MobileNetV3为模型创建出七种表情分类的模型，模型给出各种表情的预测概率，程序选取概率最大的表情作为当前识别结果。

结果显示功能就是把识别的结果反馈给用户。系统在实时画面里画出人脸框以及关键点，给出表情类别，置信度，帧率，检测耗时和分类耗时这些信息。既可以显示识别结果，又可以观察系统运行情况。

结果保存功能主要是用来保存实验记录以及效果展示的。当系统识别出比较清楚、置信度较高的画面时，可以保存一部分图像，用作以后的分析以及论文展示的资料。该功能不是识别流程的核心，但是可以用来记录系统实际运行的效果。

2.1.3 性能需求分析

端侧人脸表情识别系统除了功能完整外，还需要满足一定的性能要求。本文从识别效果、实时性、资源占用和稳定性四个方面进行考虑。

在识别效果方面，表情分类模型应在测试集上达到较稳定的识别结果。由于表情图像之间存在相似性，例如悲伤和中性、恐惧和惊讶在局部特征上容易混淆，因此评价模型时不能只看准确率，也需要结合宏平均F1值和混淆矩阵进行分析。具体训练结果和各类别表现将在第4章中说明。

在实时性方面，系统应保证视频画面能够连续刷新，检测框和表情结果能够随人脸变化而更新。树莓派端算力有限，如果每一帧都完整执行人脸检测和表情分类，容易增加处理耗时。因此，系统在总体设计时需要为后续推理优化预留空间，例如采用轻量化模型、控制检测频率和减少不必要的重复计算。

在资源占用方面，系统应避免使用过大的模型结构。本文选择YuNet和MobileNetV3，主要考虑二者相对轻量，便于在端侧环境中集成。模型转换和量化等具体部署工作放在第5章中介绍。

在稳定性方面，系统需要能够连续完成摄像头读取、人脸检测、表情分类和结果显示。对于毕业设计原型系统而言，稳定性体现在运行过程中不出现摄像头断开、窗口无响应和程序崩溃等情况。后续运行测试将围绕这些现象进行观察。

2.2 系统总体架构设计

2.2.1 系统总体架构概述

根据上述需求，本文系统采用“PC端训练 + 树莓派端部署”的总体架构，如图2-1所示。PC端负责完成模型训练、模型评估和模型格式转换；树莓派端负责加载转换后的模型，并结合摄像头完成实时识别。该架构既能利用PC端GPU提高训练效率，也能验证模型在端侧设备上的实际运行能力。

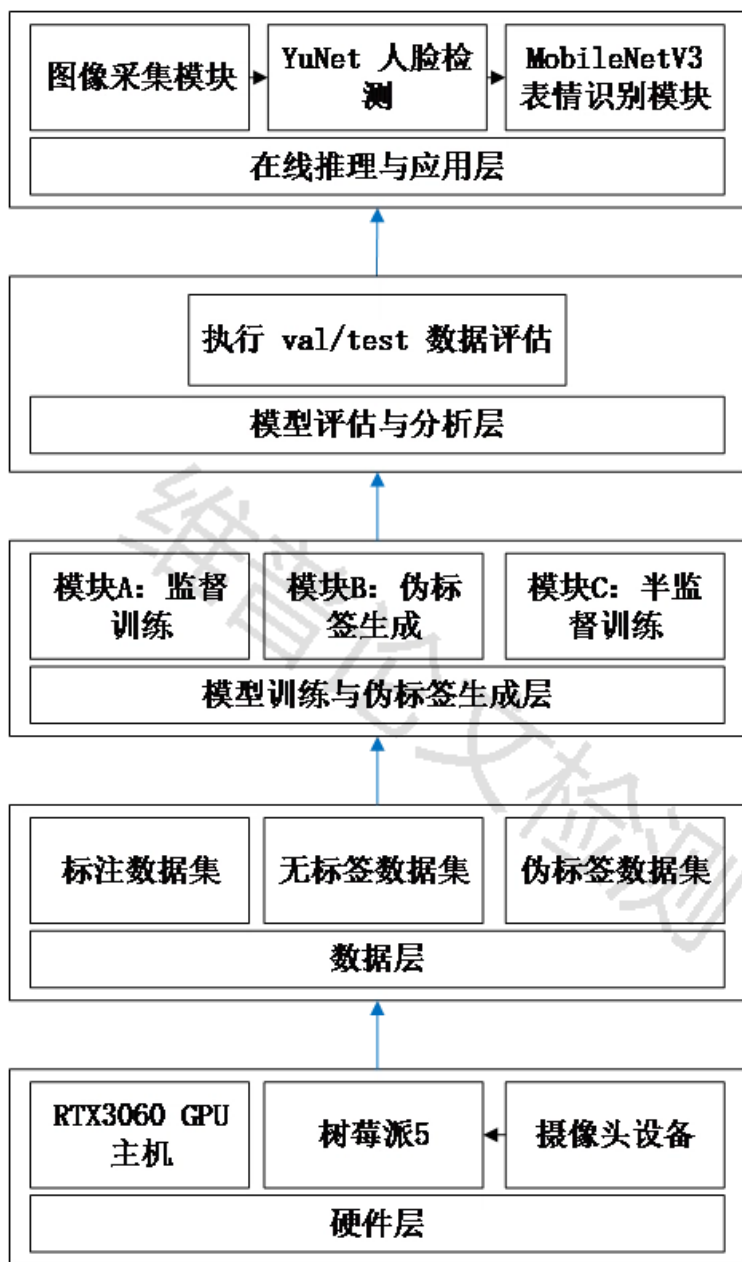


图2-1 端侧人脸表情识别系统总体技术路线图

从功能组成上看，系统可以分为数据处理部分、模型训练部分、模型转换部分和端侧推理部分。数据处理部分负责整理FER2013格式数据和扩展样本；模型训练部分负责训练MobileNetV3表情分类模型；模型转换部分负责将训练完成的模型转换为适合端侧推理的格式；端侧推理部分负责在树莓派上调用摄像头、人脸检测模型和表情分类模型，完成实时识别。

其中，数据处理和模型训练主要服务于第3章和第4章内容，模型转换和端侧推理主要服务于第5章内容。第2章只对系统整体关系进行说明，避免与后续章节重复。

2.2.2 系统识别流程设计

系统实时识别流程如图2-2所示。程序启动后，先初始化摄像头、人脸检测模型、表情分类模型和类别标签。初始化完成后，摄像头开始采集图像帧，程序对当前帧进行人脸检测。如果画面中没有检测到人脸，则继续读取下一帧；如果检测到人脸，则根据检测框截取人脸区域，并进行后续预处理。

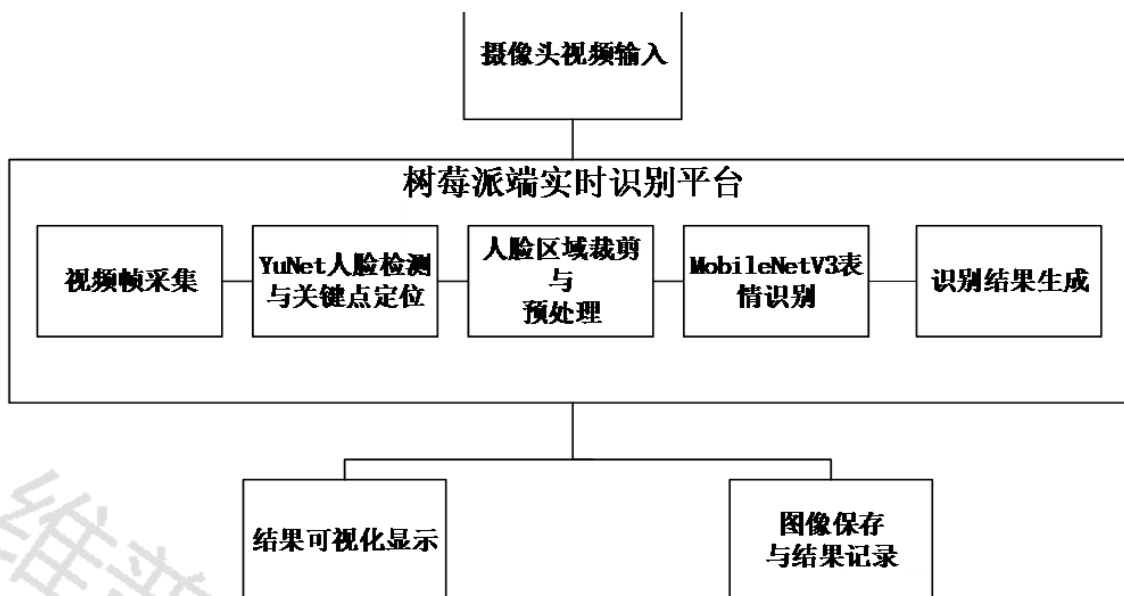


图2-2 系统识别流程图

在人脸预处理阶段，程序会对检测框进行边界检查，避免裁剪区域超出图像范围。随后，将人脸区域调整为表情分类模型要求的输入尺寸，并完成通道转换和归一化。该步骤虽然属于中间处理环节，但对分类效果影响较大，因此需要与训练阶段输入方式保持一致。

预处理完成后，分类模型输出七类表情的预测概率。程序根据最大概率确定当前表情类别，并在实时画面中绘制检测框、关键点、类别名称和置信度。为了方便调试，界面中还可以显示FPS、检测耗时和分类耗时等运行信息。部分满足条件的识别画面会被保存，用于后续效果展示和误识别分析。

2.2.3 系统功能模块划分

根据系统架构以及识别流程，本文把端侧人脸表情识别系统分为五个功能模块。

图像采集模块主要是用来读取摄像头的画面的，是系统的主要输入端。该模块要保证画面可以一直送入程序，为人脸检测、表情分类提供基础数据。

人脸检测模块可以定位视频帧中的人脸位置。该模块的输出是人脸框和关键点信息，后面预处理模块根据检测框裁剪人脸区域。

图像预处理模块是用来把人脸区域转换成模型输入格式的。该模块把人脸检测、表情分类两部分的输入整合起来，是保证训练输入和推理输入一致的关键部分。

表情识别模块主要是实现对七种表情的识别。本模块采用MobileNetV3分类模型做为骨干网络，得到的是表情类别和置信度的输出。

结果输出和管理模块主要是对识别结果进行显示以及保存。显示内容为人脸框、关键点、类别、置信度和运行状态信息，保存内容主要用作后续实验记录和论文展示。

经过以上模块划分之后，系统各个部分的职责比较清楚。图像采集、人脸检测、结果展示属于端侧系统运行过程，表情识别模型构建与训练在后面章节中会详细说明。

2.3 关键技术与方法选型

2.3.1 人脸检测模型选型

人脸检测是表情识别系统的前置环节。若检测结果不稳定，后续分类模型即使训练效果较好，也可能因为输入区域不准确而产生误判。因此，检测模型需要在速度、精度和部署便利性之间取得平衡。

本文选用YuNet作为人脸检测模型。YuNet能够输出人脸框和关键点信息，调用方式较为简洁，适合与OpenCV环境结合使用。对于本文的室内近距离识别场景而言，YuNet能够满足前端人脸定位需求，同时不会给树莓派端带来过高计算负担。该部分在系统中主要承担“找人脸”的任务，后续表情判断仍由分类模型完成。

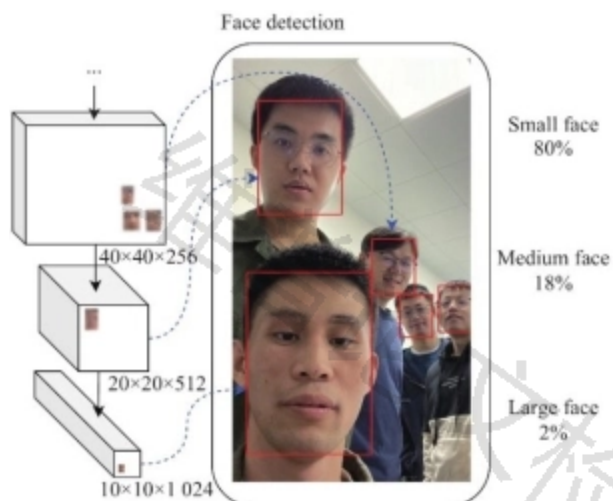


图2-3 YuNet人脸检测示意图

2.3.2 表情分类模型选型

表情分类模型需要对裁剪后的人脸图像进行七分类判断。考虑到系统最终需要部署在树莓派端，分类模型不能只追求复杂结构和高计算量，而应兼顾模型大小、推理速度和识别效果。

MobileNetV3是一种面向移动端和嵌入式设备的轻量化卷积神经网络，适合用于资源受限场景下的图像分类任务[9]。本文以MobileNetV3作为表情分类模型骨干网络，并将最终分类层调整为七类输出。这样既能利用其轻量化优势，也能够适配本文的人脸表情识别任务。

在本文系统中，MobileNetV3主要负责从人脸区域中提取表情相关特征，并输出愤怒、厌恶、恐惧、高兴、中性、悲伤和惊讶七类概率。具体模型结构调整、训练参数和训练策略将在第3章展开。

2.3.3 端侧运行方案选型

由于本文最终需要在Raspberry Pi 5上运行系统，模型格式和程序流程都需要考虑端侧设备的限制。树莓派端与PC训练环境不同，其CPU、内存和I/O资源有限，如果直接运行完整训练环境，部署过程会较复杂，运行负载也较高。

因此，本文采用“PC端训练、端侧推理”的方式。模型在PC端训练完成后，再转换为适合树莓派加载的推理格式。树莓派端程序只保留运行所需的摄像头读取、人脸检测、图像预处理、表情分类和结果显示流程。这样可以减少端侧环境配置复杂度，也便于后续进行实时性测试。

需要说明的是，模型转换、TensorFlow Lite部署和具体运行参数属于系统实现内容，本文将在第5章中详细说明。本节只从系统架构角度说明端侧运行方案的选择理由。

2.4 本章小结

本章主要对人脸表情识别系统的功能需求进行分析，并给出系统的总体架构。本文系统面向室内近距离人机交互场景，主要实现摄像头采集、人脸检测、七类表情分类、结果显示、图像保存等主要功能。系统在提高分类准确率的时候，还要考虑树莓派端运行时的实时性、资源消耗和稳定状况。

第二章给出了“PC端训练 + 树莓派端部署”的总体结构。PC端做数据处理、模型训练和格式转换，树莓派端做摄像头实时采集、人脸检测、表情分类和结果显示。该架构可以较好地支持本文从模型训练到端侧运行的全过程。最后，本章就YuNet人脸检测模型、MobileNetV3表情分类模型以及端侧运行方案进行选型说明。下一章主要讲的是表情分类模型的构建以及训练，即数据集的整理、图像预处理、模型结构的调整和训练策略的设计。

第3章 表情识别模型构建与训练

3.1 数据集构建与预处理

第2章已经完成了端侧人脸表情识别系统的需求分析和总体架构设计。本章不再展开树莓派端运行流程，而是围绕表情分类模型本身进行说明，重点介绍训练数据整理、数据集划分、图像预处理、MobileNetV3分类网络调整以及训练策略设计等内容。

在人脸表情识别任务中，训练数据的质量会直接影响模型效果。不同表情类别之间存在一定相似性，例如恐惧、悲伤和厌恶在局部表情特征上容易混淆；同一类表情在不同个体、光照和姿态下也会呈现较大差异。因此，本文在原始FER2013数据基础上进行了样本扩充、合并、清洗和重新划分，并统一整理为CSV格式，作为后续模型训练的输入数据。

3.1.1 数据集构建与扩展

本文表情分类任务以FER2013数据集为基础[4]，并在此基础上补充扩展样本。识别类别共包括七类基本表情，分别为愤怒、厌恶、恐惧、高兴、悲伤、惊讶和中性，对应类别编号为0至6。

原始FER2013数据集中，愤怒类3110张、厌恶类248张、恐惧类819张、高兴类9355张、悲伤类4370张、惊讶类4462张、中性类12905张，总计35269张。可以看出，原始数据中不同类别样本数量差异较大，其中厌恶类和恐惧类样本数量明显偏少。若直接使用该数据进行训练，模型容易偏向样本数量较多的类别。

为缓解这一问题，本文在原始数据基础上补充了扩展样本，并对数据进行统一整理。扩充后数据集总量达到410246张。为了直观展示扩充前后的变化，本文绘制了原始FER2013数据集与扩充后数据集的类别数量对比图，如图3-1所示。

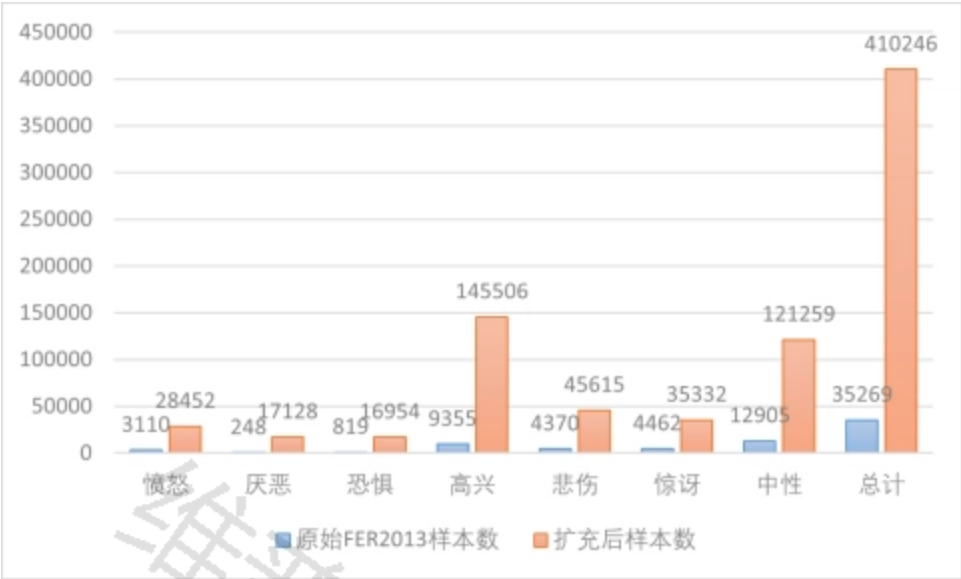


图3-1 原始FER2013与扩充后数据集类别数量对比

由图3-1可以看出，扩充后各类样本数量均有所增加。其中，原始数据中较少的厌恶类和恐惧类分别增加至17128张和16954张，数据稀缺问题得到一定缓解。与此同时，高兴类和中性类仍然是样本数量较多的类别，说明扩充后的数据集仍存在类别不平衡现象。因此，后续训练中仍需要引入类别均衡策略，避免模型过度偏向多数类别。

3.1.2 CSV数据合并与数据集划分

本文训练过程中没有直接采用文件夹路径读取方式，而是将原始样本和扩展样本统一转换为CSV格式进行管理。每条CSV记录主要包括类别标签和图像像素信息，部分扩展样本也可通过图像路径字段读取。这样处理后，不同来源的数据可以通过统一的数据加载脚本读取，减少了训练阶段对样本来源的额外判断。

在数据整理阶段，本文首先将原始训练、验证和测试数据与扩展样本进行合并，生成统一的数据文件。随后，对样本进行随机打乱，并在各类别内部进行划分，最终得到train.csv、val.csv、test.csv和unlabeled.csv四个文件。其中，训练集用于基础监督训练，验证集用于训练过程中的模型选择，测试集用于最终模型评价，无标签集则用于后续伪标签生成。

本文采用的划分方式为：先在每个类别内部预留约20%的样本作为无标签集；剩余有标签样本再按照8:1:1比例划分为训练集、验证集和测试集。对于无法完全整除的少量样本，统一并入无标签集。这样可以使训练集、验证集和测试集在各类别上保持相近比例，同时为后续伪标签训练预留数据。

数据集划分结果如表3-1所示。

表3-1 扩充后数据集划分结果

数据集	样本数量	主要用途
训练集	262536	用于基础监督训练
验证集	32817	用于训练过程中的模型选择
测试集	32817	用于最终模型性能评价
无标签集	82076	用于伪标签生成与辅助训练
合计	410246	—

各子集类别分布如表3-2所示。

表3-2 划分后各子集类别分布

表情类别	训练集	验证集	测试集	无标签集
愤怒	18208	2276	2276	5692
厌恶	10960	1370	1370	3428
恐惧	10848	1356	1356	3394
高兴	93120	11640	11640	29106
悲伤	29192	3649	3649	9125
惊讶	22608	2826	2826	7072
中性	77600	9700	9700	24259
合计	262536	32817	32817	82076

从表3-2可以看出，训练集、验证集和测试集在类别比例上基本保持一致，这有利于保证训练过程和评估过程的数据分布相近。无标签集虽然在CSV文件中保留原始类别字段，但在伪标签生成阶段并不直接使用该字段作为监督标签，而是由训练后的模型重新预测并筛选高置信度样本。

3.1.3 数据预处理与增强策略

由于MobileNetV3模型要求固定尺寸输入，本文在训练前需要对CSV中的图像数据进行还原和预处理。对于FER2013格式样本，训练脚本先将像素序列还原为48×48灰度图，再转换为三通道图像，以适配基于ImageNet预训练权重的MobileNetV3模型。随后，所有输入图像统一调整为224×224像素。

在归一化处理方面，本文采用ImageNet数据集常用的均值和标准差进行标准化处理。这样设置主要是为了与预训练模型的输入分布保持一致，减少从ImageNet预训练模型迁移到表情识别任务时的输入差异。

训练阶段采用适度数据增强，以提高模型对人脸姿态变化、轻微旋转和局部遮挡的适应能力。具体包括随机水平翻转、随机旋转、随机裁剪缩放和随机擦除等操作。其中，随机水平翻转用于模拟左右姿态变化；随机旋转用于模拟头部轻微偏转；随机裁剪缩放用于增强模型对人脸边界变化的容忍度；随机擦除则用于模拟局部遮挡情况。

验证集和测试集不使用随机增强，只进行尺寸调整和归一化处理。这样可以保证评估阶段输入稳定，避免随机增强对评价结果造成干扰。通过上述预处理方式，训练阶段能够提供更多样的输入变化，而评估阶段则保持统一的测试条件。

3.1.4 类别不平衡处理

虽然本文对数据集进行了扩充，但从图3-1和表3-2可以看出，各类别之间仍然存在数量差异。高兴类和中性类样本数量较多，厌恶类和恐惧类相对较少。如果直接采用普通交叉熵损失进行训练，模型可能更容易学习到多数类别特征，而对少数类别表情识别不足。

针对这一问题，本文在训练阶段引入类别均衡策略。训练脚本会统计训练集中各类别样本数量，并根据类别分布计算类别权重。样本数量较少的类别在损失函数中获得相对较高权重，样本数量较多的类别权重相对降低，从而

减弱类别数量差异对模型训练的影响。

此外，本文在基础训练阶段还设置了每类样本数量上限，用于控制单轮训练中多数类样本的占比。这样可以在保证训练样本规模的同时，减少多数类样本对训练过程的过度主导。类别均衡策略并不能完全消除类别间混淆，但可以提升训练过程对少数类别的关注度。

3.2 模型架构设计

3.2.1 MobileNetV3骨干网络选型

本文表情分类模型采用MobileNetV3作为骨干网络。MobileNetV3是一种面向移动端和资源受限设备设计的轻量化卷积神经网络，在模型规模、计算量和特征表达能力之间具有较好的平衡。考虑到本文系统最终需要服务于树莓派端实时识别，分类模型不能过大，因此选择MobileNetV3作为基础网络结构。

在具体实现中，本文采用MobileNetV3-large版本。相比MobileNetV3-small，large版本具有更强的特征提取能力，适合在PC端训练后再进行模型转换和端侧部署。由于人脸表情识别任务需要区分眼部、嘴部和面部纹理等细节变化，模型需要具备一定的表达能力。MobileNetV3-large能够在保持相对轻量的同时提供较好的分类基础。

本文使用ImageNet预训练权重初始化模型参数，再根据表情分类任务对输出层进行调整。这样可以利用预训练模型已有的图像特征提取能力，减少从零开始训练带来的不稳定性。

3.2.2 分类层调整与输出设计

原始MobileNetV3模型面向ImageNet分类任务，输出类别数量与本文任务不一致。因此，本文保留MobileNetV3的主干特征提取部分，将最后的全连接分类层替换为七分类输出层。调整后的模型输出维度为7，对应愤怒、厌恶、恐惧、高兴、悲伤、惊讶和中性七类表情。

模型输入为 224×224 的三通道人脸图像。输入图像经过卷积层、倒残差结构和特征聚合后，最终由分类层输出七个类别的logits。训练时，logits用于计算交叉熵损失；推理时，通过Softmax得到各类别预测概率，并选择概率最大的类别作为预测结果。

这种调整方式没有改变MobileNetV3的主体结构，只对任务相关的输出层进行替换，符合迁移学习的基本思路。对于本文任务而言，该方法能够在较小改动下将通用图像分类模型迁移到人脸表情分类任务中。

3.2.3 模型评价指标

为了评价表情分类模型的训练效果，本文主要采用准确率和宏平均F1值作为评价指标。

准确率用于衡量所有样本中被正确分类的比例，可以直观反映模型整体识别效果。但在类别分布不均衡的情况下，仅使用准确率可能会掩盖少数类识别不足的问题。例如，如果模型对样本数量较多的高兴类和中性类识别较好，即使对厌恶类或恐惧类识别一般，总体准确率仍可能较高。

因此，本文同时采用宏平均F1值作为模型选择指标。宏平均F1值会分别计算各类别的F1值，再对各类别结果取平均，更适合评价类别不均衡情况下模型对各类表情的综合识别能力。训练过程中，本文以验证集宏平均F1值作为最优模型保存依据，而不是只依据总体准确率进行模型选择。这样能够使模型在整体识别效果和各类别均衡表现之间取得更合理的平衡。

3.3 训练策略设计

本节主要说明表情分类模型的训练方案设计，重点介绍基础监督训练、伪标签辅助训练和多阶段训练流程。需

要说明的是，本节只说明训练思路和参数设置，不展开具体训练结果分析。各阶段训练过程、伪标签生成结果和最终实验结果将在第4章中进一步讨论。

3.3.1 基础监督训练方案

基础监督训练阶段只使用真实标注样本，包括训练集、验证集和测试集中的有标签数据。其中，训练集用于参数更新，验证集用于模型选择，测试集用于后续性能评价。该阶段的目标是先获得一个具有基本表情分类能力的初始模型，为后续伪标签生成提供基础。

训练时，模型输入为经过增强和归一化处理的人脸图像，标签为七类表情编号。损失函数采用带类别权重的交叉熵损失，并结合标签平滑减少模型过度自信。优化器采用AdamW，学习率使用预热和余弦衰减策略。训练过程中根据验证集宏平均F1值保存最优模型，避免只依据训练损失下降来判断模型优劣。

基础监督训练阶段不引入无标签样本，主要用于建立可靠的初始分类器。该阶段训练得到的模型将作为后续伪标签生成的教师模型。

3.3.2 伪标签辅助训练方案

在基础监督训练完成后，本文使用训练得到的模型对无标签集进行预测，并根据预测置信度筛选伪标签样本。伪标签生成时，模型对无标签样本输出七类概率，程序保留置信度达到设定阈值的样本，并将预测类别作为伪标签写入新的CSV文件。

第一轮伪标签生成使用Stage 1模型作为教师模型，筛选阈值设为0.85，并限制每类最多保留一定数量的样本，以避免某些类别伪标签过多。第二轮伪标签生成使用后续训练得到的模型重新预测无标签样本，阈值提高到0.90，以提升伪标签质量。

需要说明的是，伪标签并不等同于真实人工标注，其中可能包含预测错误样本。因此，在后续训练中，本文没有将伪标签作为核心创新点，而是将其作为辅助训练手段。通过置信度筛选、类别数量限制和分阶段引入，可以在一定程度上减少错误伪标签对模型训练的影响。

3.3.3 多阶段训练流程设计

本文训练过程分为三个阶段。各阶段训练数据和模型来源如表3-3所示。

表3-3 多阶段训练流程设计

阶段	训练数据	初始化模型	主要目标	输出模型
Stage 1	有标签训练集	ImageNet预训练模型	建立基础表情分类模型	Stage 1最优模型
Stage 2	有标签训练集 + 第一轮伪标签数据	Stage 1最优模型	利用伪标签扩展监督信息	Stage 2最优模型
Stage 3	有标签训练集 + 第二轮伪标签数据	Stage 2最优模型	更新伪标签质量并继续优化	最终分类模型

在Stage 1中，模型只使用真实标注样本进行训练，获得初始分类能力。在Stage 2中，先利用Stage 1模型对无标签集生成第一轮伪标签，再将筛选后的伪标签样本与真实标注样本共同用于训练。在Stage 3中，使用Stage 2模型重新生成伪标签，并继续进行联合训练。

这种多阶段训练流程的目的，是逐步提高无标签样本的利用效率。相比一次性加入全部无标签样本，分阶段筛选伪标签可以使模型先从较可靠的样本开始学习，再逐步更新训练数据。该流程仍属于训练优化策略，不改变 MobileNetV3 主体结构。

3.3.4 训练参数设置

为了保证训练过程具有较好的稳定性和一致性，本文在各训练阶段采用统一的基础训练参数。

本文各阶段训练的主要参数保持一致：输入尺寸为 224×224 ，batch size 设为 128，最大训练轮数为 200，初始学习率为 $5e-4$ ，最低学习率为 $1e-6$ ，优化器采用 AdamW，权重衰减设为 $1e-4$ ，标签平滑系数设为 0.04，类别均衡参数 beta 设为 0.995，早停轮数设为 20，随机种子设为 42。Stage 2 和 Stage 3 在此基础上引入伪标签样本，伪标签样本按置信度筛选后参与后续训练。

在训练稳定性方面，本文主要采用以下处理方式。首先，学习率使用线性预热和余弦衰减，避免训练初期学习率过大造成参数波动。其次，采用 AdamW 优化器和权重衰减，以降低模型过拟合风险。再次，启用自动混合精度训练，以减少显存占用并提高训练效率。最后，使用梯度裁剪限制梯度异常波动，增强训练过程稳定性。

在模型保存方面，训练过程以验证集宏平均 F1 值作为最优模型判断依据。当验证集宏平均 F1 值提升时，保存当前模型；如果连续多轮没有提升，则触发早停机制。该策略可以避免训练后期过拟合，同时保证最终保存模型具有较好的综合分类能力。

3.4 本章小结

本章围绕表情分类模型的构建与训练方案进行了说明。首先，介绍了数据集来源、扩充过程和 CSV 格式整理方式，并将扩充后数据集划分为训练集、验证集、测试集和无标签集，为后续监督训练和伪标签辅助训练提供数据基础。

其次，本章说明了图像预处理和数据增强方法。训练阶段采用随机翻转、随机旋转、随机裁剪缩放和随机擦除等操作，以提升模型对姿态变化和局部遮挡的适应能力；验证集和测试集只进行尺寸调整和归一化处理，以保证评价过程稳定。

最后，本章完成了 MobileNetV3 表情分类模型设计和训练策略说明。模型以 MobileNetV3-large 为骨干网络，将分类层调整为七类输出，并采用类别均衡、标签平滑、学习率调度、早停和伪标签辅助训练等方法提高训练稳定性。下一章将基于本章设计的训练流程，对不同阶段的训练结果和最终模型表现进行分析。

第4章 模型训练优化

4.1 训练优化思路与总体流程

在第3章完成数据集构建、模型结构调整和训练策略设计后，本章进一步对模型训练过程和模型训练结果进行分析。本文的训练目标并不是单纯追求复杂算法结构，而是在 MobileNetV3 轻量化模型基础上，通过数据扩充、类别均衡、伪标签辅助训练和多阶段优化等方式，获得能够支撑后续树莓派端部署的表情分类模型。

从整体训练流程来看，本文将模型训练分为三个阶段。第一阶段为基础监督训练，仅使用有标签训练集对 MobileNetV3-large 模型进行训练，目的是获得一个具备基本表情分类能力的初始模型。第二阶段在第一阶段模型基础上生成第一轮伪标签，并将有标签样本与伪标签样本联合用于训练，使模型能够利用无标签数据中的有效信息。第三阶段使用第二阶段模型重新生成伪标签，并继续进行联合训练，以进一步稳定模型分类能力。

其中，伪标签辅助训练的作用是补充可利用样本信息，提高无标签数据的利用效率。为了避免与第3章内容重复，本章不再展开训练参数和数据增强细节，而重点分析多阶段训练结果、最终模型评价结果以及各类别识别表现。

4.2 多阶段训练结果对比

为比较不同训练阶段对模型性能的影响，本文分别对Stage 1、Stage 2和Stage 3阶段保存的最优模型进行统一评估。评估时采用验证集和测试集作为评价数据，并统计准确率和宏平均F1值。其中，准确率反映模型总体分类正确比例，宏平均F1值能够更好地反映类别不均衡场景下模型对各类表情的综合识别能力。

多阶段训练结果如表4-1所示。

表4-1 多阶段模型评价结果对比

训练阶段	训练数据	Val Acc/%	Val Macro-F1/%	Test Acc/%	Test Macro-F1/%
Stage 1	有标签训练集	71.57	64.33	71.85	64.57
Stage 2	有标签训练集 + 第一轮伪标签数据	90.58	87.48	90.82	87.56
Stage 3	有标签训练集 + 第二轮伪标签数据	90.55	87.32	90.77	87.43

由表4-1可以看出，Stage 1阶段仅使用有标签样本训练，模型已经能够完成基本七类表情分类任务，但整体效果仍有较大提升空间。该阶段测试集准确率为71.85%，宏平均F1值为64.57%，说明基础模型虽然具备初步识别能力，但在类别分布不均衡和复杂表情样本上仍存在不足。

Stage 2阶段引入第一轮伪标签数据后，模型性能出现明显提升。测试集准确率由Stage 1的71.85%提升至90.82%，宏平均F1值由64.57%提升至87.56%。这说明在基础监督模型已经具备一定判别能力的前提下，引入高置信度伪标签样本能够有效补充训练信息，使模型学习到更加充分的表情特征。

Stage 3阶段使用Stage 2模型重新生成伪标签后继续训练，其测试集准确率为90.77%，宏平均F1值为87.43%，与Stage 2结果基本接近。该结果说明，经过Stage 2训练后，模型性能已经进入相对稳定区间，第二轮伪标签更新没有带来明显增益，但也没有造成模型性能明显下降。因此，后续系统部署采用Stage 3模型作为最终分类模型，主要是因为该模型来自完整的多阶段训练流程，能够较好地衔接模型训练、评估与部署过程。

从整体结果来看，伪标签辅助训练的主要提升集中在Stage 1到Stage 2之间。Stage 3更多体现为训练流程的进一步完善和模型效果的稳定保持。

4.3 最终模型总体评价

在完成多阶段训练后，本文选取Stage 3阶段输出的最终模型进行验证集和测试集评价。最终模型的总体评价结果如表4-2所示。

表4-2 最终模型总体评价结果

数据集	Loss	Accuracy/%	Macro-F1/%
-----	------	------------	------------

验证集	0.4054	90.55	87.32
测试集	0.3970	90.77	87.43

由表4-2可以看出，最终模型在验证集和测试集上的结果较为接近。其中，验证集准确率为90.55%，宏平均F1值为87.32%；测试集准确率为90.77%，宏平均F1值为87.43%。测试集结果略高于验证集，但二者差异很小，说明模型在当前数据划分下没有出现明显过拟合，具备较稳定的泛化表现。

从评价指标角度看，准确率超过90%说明模型能够较好地完成整体七类表情分类任务；宏平均F1值达到87%左右，说明模型并不是单纯依靠高频类别获得较高准确率，而是在各类别上也保持了较均衡的识别能力。考虑到本文最终目标是将模型部署到树莓派端进行实时识别，该模型在准确率、类别均衡表现和轻量化结构之间取得了较好的平衡，能够作为后续端侧系统部署的分类模型基础。

需要注意的是，本文的模型训练目标并不是在公开数据集上追求最高指标，而是获得适合端侧部署的稳定分类模型。因此，在最终模型选择时，除了关注测试集准确率外，还需要结合宏平均F1值、混淆矩阵和各类别表现进行综合判断。

4.4 混淆矩阵与类别识别分析

为了进一步分析最终模型在不同表情类别上的识别情况，本文绘制了测试集混淆矩阵，如图4-1所示。图中纵轴表示真实类别，横轴表示预测类别，类别编号0至6分别对应愤怒、厌恶、恐惧、高兴、悲伤、惊讶和中性。

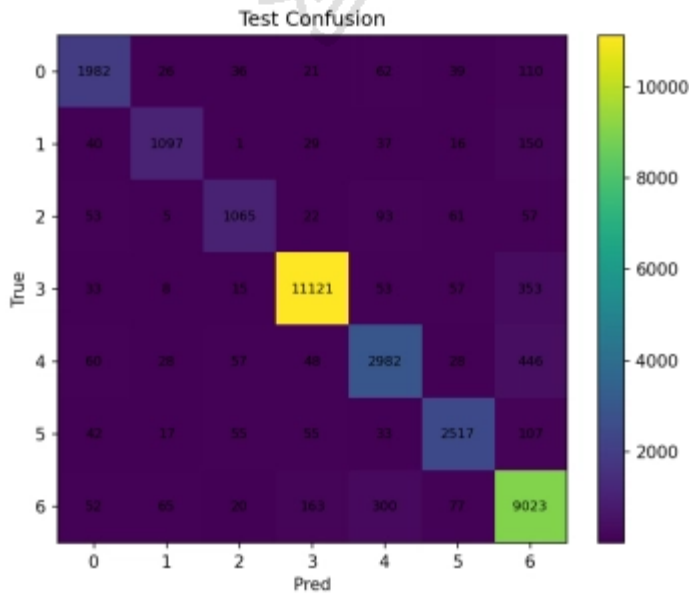


图4-1 测试集混淆矩阵

从图4-1可以看出，大部分样本集中在混淆矩阵的主对角线上，说明模型对各类别均具备较好的识别能力。其中，高兴类和中性类样本数量较多，且主对角线位置数值明显较高，表明模型对这两类表情具有较强的识别能力。高兴类共有11121个样本被正确识别，中性类共有9023个样本被正确识别。

为了进一步量化不同类别的表现，本文统计了测试集上各类别的精确率、召回率和F1值，结果如表4-3所示。

表4-3 测试集各类别识别结果

类别编号	表情类别	Precision/%	Recall/%	F1/%
0	愤怒	87.62	87.08	87.35
1	厌恶	88.04	80.07	83.87
2	恐惧	85.27	78.54	81.77
3	高兴	97.05	95.54	96.29
4	悲伤	83.76	81.72	82.73
5	惊讶	90.05	89.07	89.56
6	中性	88.06	93.02	90.47

由表4-3可以看出，高兴类的识别效果最好，其Precision、Recall和F1值分别为97.05%、95.54%和96.29%。这说明高兴表情的视觉特征较明显，例如嘴角上扬、面部肌肉变化较突出，模型较容易学习到该类表情的判别特征。中性类的F1值为90.47%，召回率达到93.02%，说明模型对中性表情也具有较好的识别能力。惊讶类F1值为89.56%，整体表现同样较稳定。

相比之下，恐惧类、悲伤类和厌恶类的F1值相对较低。其中，恐惧类F1值为81.77%，是七类表情中最低的一类；悲伤类F1值为82.73%，厌恶类F1值为83.87%。这说明这些类别在视觉表现上更容易与其他类别产生混淆。例如，恐惧表情在面部局部特征上可能与惊讶、悲伤或中性存在相似性；悲伤表情在表情强度较弱时容易接近中性；厌恶类样本数量相对较少，且面部表现差异较大，因此识别难度也较高。

结合混淆矩阵进一步观察可以发现，部分样本容易被预测为中性类。例如，真实悲伤类中有446个样本被误判为中性，真实高兴类中有353个样本被误判为中性，真实厌恶类中有150个样本被误判为中性，真实愤怒类中也有110个样本被误判为中性。这说明在表情强度较弱或面部特征不明显时，模型倾向于将样本归入中性类别。另一方面，中性类也存在被误判为悲伤和高兴的情况，其中有300个中性样本被预测为悲伤，163个中性样本被预测为高兴。这表明中性类别与弱表情类别之间仍存在一定边界模糊问题。

总体来看，最终模型在七类表情上均取得了可用的识别效果，尤其在高兴、中性和惊讶等类别上表现较好；但对于恐惧、悲伤和厌恶等类别，仍存在一定程度的混淆。该结果符合人脸表情识别任务本身的特点，也为后续模型改进提供了方向。后续可以通过进一步提高少数类样本质量、增强弱表情样本标注一致性、引入更加精细的人脸区域特征或注意力机制等方式，改善易混类别的识别效果。

4.5 本章小结

本章围绕表情分类模型的训练优化过程和实验结果进行了分析。首先，本文对Stage 1、Stage 2和Stage 3三个阶段的训练结果进行了对比。结果表明，仅使用有标签数据进行基础监督训练时，模型能够完成初步分类任务，但整体性能有限；在Stage 2中引入高置信度伪标签后，模型准确率和宏平均F1值均得到明显提升；Stage 3在第二轮伪标签更新后与Stage 2结果基本接近，说明模型性能已经趋于稳定。

其次，本文对最终Stage 3模型进行了总体评价。最终模型在验证集上的准确率为90.55%，宏平均F1值为87.32%；在测试集上的准确率为90.77%，宏平均F1值为87.43%。验证集和测试集结果差异较小，说明模型具备较稳定的分类能力。

最后，通过测试集混淆矩阵和各类别指标分析可以看出，模型对高兴、中性和惊讶等类别识别效果较好，但对恐惧、悲伤和厌恶等类别仍存在一定混淆。总体而言，最终表情分类模型已经能够满足后续树莓派端部署和实时识别系统测试的基本要求。下一章将在本章训练结果基础上，进一步介绍模型转换、树莓派端部署和系统运行测试过程。

第5章 系统实现与运行测试

5.1 系统实现与运行流程

前几章已经完成了端侧人脸表情识别系统的需求分析、模型构建和训练优化。第4章实验结果表明，最终Stage 3模型在测试集上取得了较稳定的识别效果，因此本章进一步围绕模型转换、树莓派端部署和实际运行测试展开说明。

本文系统最终目标不是停留在PC端离线分类实验，而是将训练完成的表情分类模型部署到树莓派端，结合摄像头输入和YuNet人脸检测模型，实现实时人脸表情识别。系统运行时，摄像头采集视频画面，程序调用YuNet检测人脸区域和关键点，然后根据检测框裁剪人脸图像，将其输入TFLite表情分类模型，最后在显示界面中绘制人脸框、关键点、表情类别、置信度、FPS和耗时信息。

在整体实现上，PC端主要承担模型训练、模型导出和格式转换任务；树莓派端主要承担视频采集、人脸检测、表情分类、结果显示和图像保存任务。通过这种方式，本文完成了从模型训练、模型转换到树莓派端运行验证的完整闭环。本章重点通过模型导出文件、端侧推理参数和实际运行截图说明系统实现过程。

5.2 软件环境与硬件平台配置

5.2.1 硬件平台配置

本文系统涉及PC端训练转换平台和树莓派端部署平台。PC端用于数据处理、模型训练、模型评估和模型格式转换；树莓派端用于部署TFLite模型并完成实时识别测试。

硬件平台配置如表5-1所示。

表5-1 硬件平台配置

平台	配置项	具体信息
PC端训练与转换平台	处理器	Intel Core i7-10870H @ 2.20GHz
PC端训练与转换平台	内存	32.0 GB
PC端训练与转换平台	GPU	NVIDIA RTX3060
端侧部署平台	开发板	Raspberry Pi 5
端侧部署平台	摄像头	H052-8614
端侧部署平台	摄像头采集分辨率	640×480

PC端硬件主要用于完成多阶段模型训练和模型转换任务。树莓派5则作为最终部署平台，用于验证模型在端侧环境下的实际运行效果。由于树莓派平台算力和内存资源有限，部署阶段需要控制模型大小、推理频率和单帧处理负载。

5.2.2 软件环境与工具

本文模型训练阶段主要使用PyTorch完成MobileNetV3表情分类模型训练；模型转换阶段使用ONNX、onnxruntime、onnxsim、onnx2tf和TensorFlow Lite等工具；树莓派端使用OpenCV、YuNet和TFLite运行时完成实时推理。模型导出脚本中设置最终导出模型类别数为7，模型名称为MobileNetV3-large，加载的权重文件为best_model_stage3.pth，导出输入尺寸为224，并采用NCHW输入布局。软件工具及用途如表5-2所示。

表5-2 软件工具与用途

软件或工具	主要用途
Python	训练、模型转换和树莓派端推理程序运行环境
PyTorch	加载和训练MobileNetV3表情分类模型
ONNX	作为PyTorch模型到其他框架之间的中间格式
onnxsim	对ONNX模型进行简化
onnxruntime	对导出的ONNX模型进行快速验证
TensorFlow / onnx2tf	将ONNX模型转换为TensorFlow SavedModel
TensorFlow Lite / tflite-runtime	在树莓派端执行TFLite模型推理
OpenCV	摄像头读取、YuNet检测、图像绘制和窗口显示

5.3 模型导出与端侧部署

5.3.1 模型导出与格式转换

第4章确定Stage 3模型作为最终部署模型。该模型最初以PyTorch权重文件形式保存，不能直接在树莓派端高效运行，因此需要将其转换为适合端侧部署的TFLite模型。

本文模型转换过程包括以下步骤。首先，加载训练得到的best_model_stage3.pth权重文件，并构建输出类别数为7的MobileNetV3-large模型。其次，将PyTorch模型导出为ONNX格式。再次，使用ONNX简化工具生成简化后的ONNX模型。随后，将简化后的ONNX模型转换为TensorFlow SavedModel格式。最后，使用TensorFlow Lite转换器生成TFLite模型。导出脚本中量化方式设置为FP16，验证样本数量设置为200，用于检查转换后模型输出的一致性。模型转换后得到的主要文件如表5-3所示。

表5-3 模型导出文件

文件	类型	大小	作用
model.onnx	ONNX文件	16445 KB	PyTorch导出的ONNX中间模型
model_simplified.onnx	ONNX文件	16459 KB	简化后的ONNX模型
saved_model	文件夹	—	TensorFlow SavedModel目录
saved_model.pb	PB文件	16570 KB	SavedModel模型文件
model_simplified_float32.tflite	TFLite文件	16451 KB	float32格式TFLite模型
model_fp16.tflite	TFLite文件	8265 KB	FP16量化后的TFLite模型

由表5-3可以看出, FP16量化后的 model_fp16.tflite 文件大小约为8265 KB, 而float32格式TFLite模型约为16451 KB。FP16量化后模型文件体积约减少一半[14][15], 能够降低树莓派端存储占用, 也更适合端侧部署。

5.3.2 模型转换一致性验证

模型格式转换可能导致输出结果发生偏移, 因此在部署前需要进行一致性验证。本文使用转换脚本对PyTorch、ONNX和TFLite三种形式的模型输出进行对比, 验证样本数量为200。验证报告显示, PyTorch与ONNX的平均最大绝对误差约为 2.50×10^{-6} , PyTorch与TFLite的平均最大绝对误差约为0.0068, ONNX与TFLite的平均最大绝对误差也约为0.0068; 三种模型在验证样本上的预测一致性均为1.0。

模型转换一致性验证结果如表5-4所示。

表5-4 模型转换一致性验证结果

对比项	平均最大绝对误差	平均绝对误差
PyTorch与ONNX	2.50×10^{-6}	1.18×10^{-6}
PyTorch与TFLite	0.0068	0.0033
ONNX与TFLite	0.0068	0.0033

从表5-4可以看出, PyTorch模型与ONNX模型之间误差极小, 说明ONNX导出过程基本保持了模型输出一致性。TFLite模型由于采用FP16量化, 输出与原PyTorch模型之间存在小幅数值差异, 但差异范围较小, 预测结果保持一致。因此, 本文认为该TFLite模型能够作为树莓派端部署模型使用。

5.4 树莓派端推理程序设计

5.4.1 推理程序总体结构

树莓派端推理程序主要包括摄像头读取、YuNet人脸检测、目标跟踪、人脸区域裁剪、TFLite表情分类、结果绘制和图像保存等部分。程序启动后, 首先加载TFLite表情分类模型和YuNet人脸检测模型, 然后初始化摄像头并进入实时循环。

树莓派端推理程序加载FP16量化后的model_fp16.tflite作为表情分类模型, 并加载face_detection_yunet_2023mar.onnx作为YuNet人脸检测模型。程序中的表情类别标签依次为anger、disgust、fear、happy、sad、surprise和neutral, 输入图像尺寸设置为224, 并使用与训练阶段一致的ImageNet均值和标准差进行归一化处理。

推理程序主要流程如下:

第一, 摄像头读取视频帧。程序通过异步摄像头读取类获取实时图像, 避免摄像头读取阻塞主循环。

第二, YuNet检测人脸。程序对缩放后的输入图像执行人脸检测, 得到检测框和五点关键点。

第三, 目标跟踪与位置更新。对于非检测帧, 程序利用上一轮检测结果维持目标状态, 减少重复检测开销。

第四, 人脸区域裁剪。程序根据检测框扩展人脸区域, 并裁剪出分类模型输入图像。

第五, TFLite模型推理。裁剪后的人脸图像经过RGB转换、缩放和归一化后输入TFLite模型, 输出七类表情概率。

第六, 结果绘制与保存。程序在画面中绘制检测框、关键点、预测类别、置信度、FPS和耗时信息, 并异步保存高置

信度图像。

5.4.2 树莓派端运行参数

树莓派平台算力有限，如果对每一帧图像都执行完整检测和分类，容易造成画面卡顿。因此，本文在端侧推理程序中设置了多项运行参数，以控制检测和分类负载。

主要运行参数如表5-5所示。

表5-5 树莓派端推理参数设置

参数	设置值	作用
摄像头分辨率	640×480	获取实时输入画面
处理分辨率	640×480	主程序处理图像尺寸
YuNet检测输入尺寸	256×192	降低人脸检测计算量
检测间隔	detect_every=3	每3帧执行一次完整检测
分类间隔	infer_every=2	避免连续帧重复分类
YuNet置信度阈值	0.7	过滤低置信度检测框
NMS阈值	0.3	去除重复检测框
TFLite线程数	4	提高分类推理效率
表情置信度阈值	0.45	过滤低置信度分类结果
人脸框扩展比例	0.18	保留较完整的人脸区域
最大跟踪人脸数	4	控制画面中最多跟踪目标
单帧最多分类人脸数	1	降低多目标分类负载
目标帧率参数	30 FPS	控制显示刷新节奏

在人脸检测部分，程序将检测输入尺寸设置为256×192，检测结束后再将检测框和关键点映射回原始画面尺寸。这样可以减少YuNet检测阶段的计算量，同时保证画面中检测框位置正确。

在表情分类部分，程序只对主要人脸进行分类推理。虽然程序最多可跟踪4个人脸，但单帧最多只对1个人脸执行表情分类，这一设置能够降低多目标场景下的计算压力。对于毕业设计中的室内单人演示场景，该策略能够在识别功能和实时性之间取得较好的平衡。

5.5 系统运行测试与效果分析

5.5.1 功能测试

为验证系统各模块能否正常运行，本文对摄像头采集、人脸检测、表情分类、结果显示、图像保存和多人场景显示等功能进行了测试。

从功能测试结果看，系统已经能够完成从摄像头输入到识别结果显示的完整流程。YuNet模块能够检测人脸并输出关键点，TFLite分类模型能够输出表情类别和置信度，结果显示模块能够实时叠加运行信息，图像保存模块能够保存部分识别效果较好的画面。

5.5.2 实时性观察

由于本文未单独使用日志系统持续记录每一帧的性能数据，因此本节实时性结果主要来自系统运行界面中显示的FPS、延迟、检测耗时、分类耗时和主循环耗时。测试场景为室内近距离人脸表情识别，显示方式为VNC远程界面。

从运行截图可以观察到，系统在单人场景下FPS通常保持在约47~57之间，部分画面中FPS显示为52.5、53.6、53.3和56.9；在多人场景示例中，FPS约为50.9。YuNet检测耗时多处于约6~14 ms范围内，说明轻量化检测模型和低分辨率检测输入能够有效降低检测开销。表情分类耗时相对检测耗时更高，多数情况下约为18~36 ms，个别画面中达到60 ms以上，说明TFLite表情分类仍是树莓派端推理流程中的主要耗时部分。不同画面中的延迟存在一定波动，主要与摄像头读取、检测与分类推理负载有关。

系统实时性观察结果如表5-6所示。

表5-6 系统实时性观察结果

指标	观察结果
摄像头采集分辨率	640×480
目标显示帧率	30 FPS
运行界面FPS	约47~57 FPS
YuNet检测耗时	约6~14 ms
表情分类耗时	约18~36 ms
测试场景	室内近距离单人及少量多人场景

从表5-6可以看出，系统在室内测试环境下能够保持较连续的画面显示，检测框和识别类别能够随人脸状态变化进行更新。YuNet检测耗时较低，主要得益于检测输入尺寸被降低至256×192以及间隔检测策略。分类耗时相对检测耗时更高，说明TFLite表情分类仍然是树莓派端主要计算开销之一。

5.5.3 运行效果展示

为了展示系统实际运行效果，本文选取了若干树莓派端运行截图，覆盖愤怒、恐惧、高兴、中性、悲伤等单人表情识别场景，以及少量多人识别场景。运行效果如图5-1和图5-2所示。



图5-1 单人场景下多类别表情识别效果

图5-1展示了单人场景下的多类别表情识别效果。可以看出，系统能够在画面中绘制人脸检测框和五点关键点，并在检测框附近显示目标编号、预测类别和置信度。画面左上角同时显示FPS、延迟、检测耗时、分类耗时和主循环耗时等信息，便于观察系统实时运行状态。

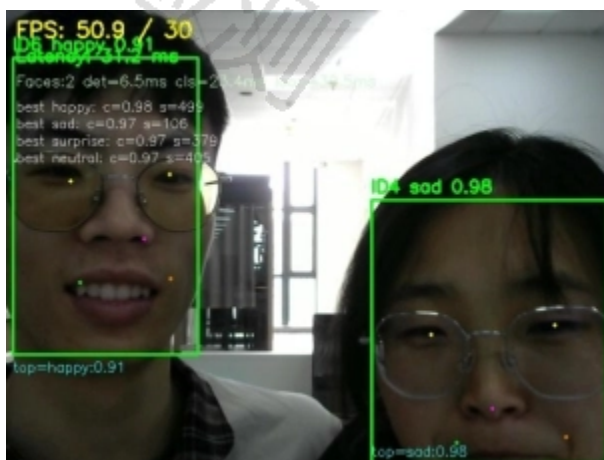


图5-2 多人场景下表情识别效果

图5-2展示了多人场景下的识别效果。当画面中出现两个人脸时，系统能够检测并绘制多个人脸框。由于程序设置了`max_infer_faces=1`，系统优先对主要目标进行表情分类，以降低多人场景下的分类推理负载。因此，当前系统更适合单人近距离实时识别场景，在多人场景下仍需要进一步优化多目标分类策略。

5.5.4 稳定性测试

为验证系统连续运行能力，本文在树莓派端对系统进行了稳定性观察。测试过程中，系统持续执行摄像头读取、人脸检测、表情分类、结果显示和图像保存等操作，观察程序是否出现崩溃、摄像头中断、界面卡死或结果无法更新等异常情况。

测试结果表明，在室内近距离场景下，系统能够较稳定地完成实时采集、检测、分类和显示任务。异步摄像头读取和异步图像保存机制对系统稳定性有一定帮助，使视频读取和图像写入不会明显阻塞主循环。

不过，当前稳定性测试主要基于人工观察，测试时长和场景复杂度仍然有限。后续可以进一步增加长时间运行测试和多场景测试，例如弱光、逆光、多人移动、侧脸和遮挡等情况，以更全面地评价系统稳定性。

5.6 本章小结

本章围绕人脸表情识别系统的实现、模型部署和运行测试进行了说明。首先，本文介绍了PC端与树莓派端的分工关系，PC端主要完成模型训练、导出和转换，树莓派端主要完成摄像头输入、YuNet人脸检测、TFLite表情分类和结果显示。

其次，本文说明了模型从PyTorch权重文件转换为ONNX、TensorFlow SavedModel和TFLite模型的过程。最终部署模型采用FP16量化后的 `model_fp16.tflite`，文件大小约为8265 KB，相比float32格式TFLite模型明显减小。模型转换一致性验证结果表明，PyTorch、ONNX和TFLite模型在验证样本上的预测结果保持一致，说明转换后的模型可以用于端侧部署。

最后，本文对树莓派端系统进行了功能测试、实时性观察、运行效果展示和稳定性测试。测试结果表明，系统能够在室内近距离场景下完成摄像头采集、人脸检测、表情分类、结果显示和图像保存等功能，运行画面较为连续，连续运行过程中未观察到程序崩溃、摄像头中断或界面卡死等异常情况。总体来看，本文系统已经实现了从模型训练、格式转换到树莓派端实时运行的完整工程闭环。

第6章 结论与展望

6.1 结论

本毕业设计围绕端侧人脸表情识别系统的设计与实现展开，完成了从数据集构建、模型训练、模型转换到树莓派端实时运行的完整流程。系统以“轻量化人脸检测 + 轻量化表情分类 + 树莓派端部署”为主要思路，前端采用YuNet完成人脸检测与关键点定位，后端采用MobileNetV3-large构建七类表情分类模型，最终在树莓派平台上实现了摄像头采集、人脸检测、表情分类、结果显示和图像保存等功能。

在模型训练方面，本文以FER2013数据集为基础，对样本进行了扩充、合并、清洗和重新划分，并统一转换为CSV格式进行训练。针对表情类别分布不均衡的问题，训练过程中采用了数据增强、类别均衡、标签平滑和伪标签辅助训练等方法。实验结果表明，最终Stage 3模型在测试集上取得了90.77%的准确率和87.43%的宏平均F1值，能够较好地完成愤怒、厌恶、恐惧、高兴、悲伤、惊讶和中性七类表情识别任务。其中，高兴、中性和惊讶等类别识别效果较好，恐惧、悲伤和厌恶等类别仍存在一定混淆。

在系统实现方面，本文完成了PyTorch模型到ONNX、TensorFlow SavedModel和TensorFlow Lite模型的转换，并采用FP16量化方式生成适合端侧运行的TFLite模型。量化后的`model_fp16.tflite` 文件体积约为8265 KB，相比float32格式模型明显减小。模型转换后，本文还对PyTorch、ONNX和TFLite模型输出进行一致性验证，保证转换后的模型能够用于端侧部署。

在树莓派端部署方面，系统集成了YuNet人脸检测、TFLite表情分类、实时画面显示和异步图像保存等功能。为了降低端侧计算负载，系统设置了较低的人脸检测输入分辨率，并采用间隔检测、间隔分类、限制单帧分类人脸数和异步读取等方式提高运行连续性。实际运行结果表明，系统在室内近距离场景下能够较稳定地完成实时识别任务，运行画面较为连续，检测框、关键点、表情类别和置信度能够正常显示，连续运行过程中未观察到程序崩溃、摄像头中断或界面卡死等异常情况。

总体来看, 本文完成了一个可运行的端侧人脸表情识别原型系统。该系统不是单纯停留在离线模型训练层面, 而是实现了从模型训练到树莓派端部署运行的工程闭环。

6.2 展望

虽然本文系统已经完成基本功能并能够在树莓派端运行, 但仍存在一些不足, 需要在后续工作中进一步改进。

第一, 模型对部分类别的识别能力仍有提升空间。从实验结果可以看出, 恐惧、悲伤和厌恶等类别的F1值相对较低, 且容易与中性、惊讶等类别发生混淆。后续可以进一步补充少数类和弱表情样本, 提高数据标注一致性, 也可以尝试引入更精细的注意力机制或面部局部区域特征, 以改善易混类别的识别效果。

第二, 系统当前更适合室内近距离单人场景。在多人场景下, 程序虽然能够检测多个人脸, 但为了保证实时性, 当前主要对主要目标进行表情分类, 无法同时对所有人脸进行高频分类。后续可以结合更高效的跟踪策略和批量推理方式, 提升多人场景下的识别能力。

第三, 端侧性能测试还可以进一步完善。本文实时性结果主要来自运行界面观察, 虽然能够反映系统基本运行状态, 但还缺少长时间自动化日志统计。后续可以在程序中加入性能记录模块, 持续统计FPS、检测耗时、分类耗时、CPU占用率和内存占用情况, 从而更准确地评估系统在不同场景下的运行表现。

第四, 系统还可以进一步优化部署方式。当前模型采用FP16 TFLite格式运行, 后续可以继续尝试INT8量化、模型剪枝或硬件加速推理方式, 以进一步降低模型体积和推理耗时。同时, 也可以针对树莓派端运行环境优化摄像头读取、显示刷新和图像保存逻辑, 提高系统整体稳定性和响应速度。

综上, 本文系统已经完成端侧人脸表情识别的基本设计与实现, 后续可从数据质量、模型结构、多目标识别和端侧性能优化等方面继续改进, 使系统在更复杂的实际场景中具有更好的适应能力。

参考文献

- [1] 蒋斌, 崔晓梅, 等. 轻量级网络在人脸表情识别上的新进展[J]. 计算机应用研究, 2024, 41(3): 663-670.
- [2] Ullah S, Ou J, et al. Facial expression recognition (FER) survey: a vision, architectural elements, and future directions[J]. PeerJ Computer Science, 2024, 10: e2024.
- [3] Kopalidis T, Solachidis V, et al. Advances in facial expression recognition: a survey of methods, benchmarks, models, and datasets[J]. Information, 2024, 15(3): 135.
- [4] Goodfellow I J, Erhan D, et al. Challenges in representation learning: a report on three machine learning contests[C]. Berlin: Springer, 2013: 117-124.
- [5] 赵晓, 杨晨, 等. 基于注意力机制ResNet轻量网络的面部表情识别[J]. 液晶与显示, 2023, 38(11): 1503-1510.
- [6] 左义海, 白武尚, 等. NCA-MobileNet: 一种轻量化人脸表情识别方法[J]. 液晶与显示, 2024, 39(4): 522-531.
- [7] 李艳秋, 李胜赵, 等. 轻量型Swin Transformer与多尺度特征融合相结合的人脸表情识别方法[J]. 光电工程, 2025, 52(1): 240234.
- [8] Jiang B, Li N, et al. Research on facial expression recognition algorithm based on improved MobileNetV3[J]. EURASIP Journal on Image and Video Processing, 2024, 2024: 22.

- [9]Howard A, Sandler M, et al. Searching for MobileNetV3[C]. Seoul: IEEE, 2019: 1314-1324.
- [10]Wu W, Peng H, Yu S. YuNet: a tiny millisecond-level face detector[J]. Machine Intelligence Research, 2023, 20(5): 656-665.
- [11]李宇豪, 吕晓琪, 等. 基于改进S3FD网络的人脸检测算法[J]. 激光技术, 2021, 45(6): 722-728.
- [12]Mohammadi M, Smith H, et al. Facial expression recognition at the edge: CPU vs GPU vs VPU vs TPU [C]. New York: ACM, 2023: 243-248.
- [13]Liu S, Ha D S, et al. Efficient neural networks for edge devices[J]. Computers & Electrical Engineering, 2021, 92: 107121.
- [14]Weng O. Neural network quantization for efficient inference: a survey[J]. arXiv preprint, 2021.
- [15]Wei L, Zhang Z, et al. Advances in the neural network quantization: a survey[J]. Applied Sciences, 2024, 14(17): 7445.

致谢

时光匆匆，大学阶段的学习生活即将结束。回顾本次毕业设计从选题、资料查阅、系统设计、模型训练到最终部署测试的整个过程，我在专业知识、实践能力和问题解决能力等方面都得到了较大的锻炼和提升。在此，我谨向所有给予我指导、帮助和支持的老师、同学以及家人表示诚挚的感谢。

首先，衷心感谢我的指导教师康莉莉在毕业设计全过程中给予的耐心指导。从课题方向确定、系统方案设计，到毕业设计结构调整、实验结果分析和格式规范修改，老师都提出了许多宝贵意见，使我能够不断发现问题并及时改进。老师严谨认真的治学态度和负责细致的指导方式，使我受益匪浅。

其次，感谢学院各位老师在本科阶段对我的培养。通过四年的课程学习和实践训练，我逐步掌握了人工智能、计算机视觉、程序设计和系统开发等方面的基础知识，也为本次毕业设计的完成奠定了重要基础。

同时，感谢同学和朋友们在毕业设计过程中给予的帮助与鼓励。在系统调试、模型训练、资料整理和毕业设计修改过程中，他们的交流与建议帮助我更好地理清思路，也让我在遇到困难时能够保持信心。

最后，感谢我的家人一直以来的理解、支持和陪伴。正是他们在学习和生活中的鼓励，使我能够顺利完成本科阶段的学习任务。

本次毕业设计仍有许多不足之处，但它也是我本科阶段一次重要的综合实践。今后我将继续保持学习和探索的态度，不断提升自己的专业能力和实践能力。

附录

```
20 # =====
21 # Transforms
22 # =====
23 def get_labeled_transforms(split: str = "train", img_size: int = IMG_SIZE) -> transforms.Compose:
24     """给有标签样本的增强，训练适度几何变化，评估仅做归一化"""
25     if split == "train":
26         return transforms.Compose([
27             transforms.Resize((img_size, img_size)),
28             transforms.RandomHorizontalFlip(p=0.5),
29             transforms.RandomRotation(15),
30             transforms.RandomResizedCrop(img_size, scale=(0.9, 1.0)),
31             transforms.ToTensor(),
32             transforms.Normalize(mean=IMAGENET_MEAN, std=IMAGENET_STD),
33             transforms.RandomErasing(p=0.20, scale=(0.02, 0.12), ratio=(0.3, 3.3)),
34         ])
35     else:
36         return transforms.Compose([
37             transforms.Resize((img_size, img_size)),
38             transforms.ToTensor(),
39             transforms.Normalize(mean=IMAGENET_MEAN, std=IMAGENET_STD),
40         ])
```

```

41
42
43 def get_weak_transforms(img_size: int = IMG_SIZE) -> transforms.Compose:
44     """无标签 weak: 尽量稳定语义, 用于产生伪标签"""
45     return transforms.Compose([
46         transforms.Resize((img_size, img_size)),
47         transforms.RandomHorizontalFlip(p=0.5),
48         transforms.ToTensor(),
49         transforms.Normalize(mean=IMAGENET_MEAN, std=IMAGENET_STD),
50     ])
51
52
53 def get_strong_transforms(img_size: int = IMG_SIZE) -> transforms.Compose:
54     """无标签 strong: 更强的几何/局部扰动以正则化"""
55     return transforms.Compose([
56         transforms.Resize((img_size, img_size)),
57         transforms.RandomResizedCrop(img_size, scale=(0.80, 1.00)),
58         transforms.RandomHorizontalFlip(p=0.5),

```

```

68 # =====
69 # Dataset helpers
70 # =====
71 def _load_fer_pixels(pixels: str) -> np.ndarray:
72     """从 FER2013 CSV 的像素串还原 48x48 灰度图"""
73     arr = np.fromstring(pixels, dtype=np.uint8, sep=' ')
74     if arr.size != 48 * 48:
75         # 奇惜: 若像素数异常, 尽量 reshape 成接近方阵
76         side = int(np.sqrt(max(arr.size, 1)))
77         side = max(8, side)
78         padded = np.zeros(side * side, dtype=np.uint8)
79         n = min(arr.size, side * side)
80         padded[:n] = arr[:n]
81         arr = padded.reshape(side, side)
82     else:
83         arr = arr.reshape(48, 48)
84     return arr
85
86
87 def _to_pil(img_arr: np.ndarray) -> Image.Image:
88     """灰度转 3 通道 PIL [0] 与 ImageNet 训练时保持一致的接口"""
89     if img_arr.ndim == 2:
90         img_arr = np.stack([img_arr] * 3, axis=-1)
91     return Image.fromarray(img_arr)
92
93
94 static_cache = {}
95 def _discover_csvs(csv_base: str) -> Tuple[str, str, str, Optional[str]]:
96     """在 csv_base 目录下自动探测 train/val/test/unlabeled 文件"""
97     assert os.path.isdir(csv_base), f"csv_base not found: {csv_base}"
98     files = [f for f in os.listdir(csv_base) if f.lower().endswith(".csv")]
99     if not files:
100         raise FileNotFoundError(f"No CSV files in {csv_base}")
101
102     def pick(preds):
103         cand = [f for f in files if any(p in f.lower() for p in preds)]
104         if not cand:
105             return None
106         # 优先精确名字
107         for name in ("train.csv", "val.csv", "validation.csv", "test.csv", "publictest.csv", "privatetest.csv", "unlabeled.csv"):

```

```

149 # =====
150 # Dataset
151 # =====
152 class FER2013Hybrid(Dataset):
153     """
154     兼容两种 CSV 格式:
155     A) 经典 FER2013 列含 "emotion", "pixels", "Usage"
156     B) 扩展格式: 列含 "label/emotion" 与 "path/filepath/image" (外部图片)
157     参数:
158     - split: 'train'|'val'|'publictest'|'test'|'unlabeled'
159     - two_views: True 时返回 (weak, strong, label) 以用于 fixmatch
160     - include_label: 无标签集设 False, 则 label 永远为 -1
161     """
162     def __init__(
163         self,
164         csv_path: str,
165         img_root: Optional[str],
166         split: str,
167         img_size: int = IMG_SIZE,
168         two_views: bool = False,
169         include_label: bool = True,
170     ) -> None:
171         self.csv_path = csv_path
172         self.img_root = img_root
173         self.split = split.lower()
174         self.img_size = img_size
175         self.two_views = two_views
176         self.include_label = include_label
177
178         # ---- 替换 FER2013Hybrid.__init__ 里"读取 CSV"这段 ----
179         self.samples: List[Dict[str, Any]] = []
180         if not os.path.exists(csv_path):
181             raise FileNotFoundError(f"CSV not found: {csv_path}")
182
183         with open(csv_path, 'r', encoding='utf-8') as f:
184             reader = csv.DictReader(f)
185             # 如果没有 Usage 列, 则整份 CSV 都视为当前 split
186             fieldnames = [fn.lower() for fn in (reader.fieldnames or [])]
187             has_usage = ("usage" in fieldnames)

```

```

291 # =====
292 # Dataloaders
293 # =====
294 def _make_loader(
295     ds: Dataset,
296     batch_size: int,
297     shuffle: bool,
298     num_workers: int = 4,
299     pin_memory: bool = True,
300     persistent_workers: bool = False,
301     prefetch_factor: int = 2,
302     drop_last: bool = False,
303 ):
304     # Windows 下 persistent_workers 只有在 num_workers>0 时才允许
305     if num_workers <= 0:
306         persistent_workers = False
307     kwargs = dict(
308         batch_size=batch_size,
309         shuffle=shuffle,
310         num_workers=num_workers,
311         pin_memory=pin_memory,
312         drop_last=drop_last,
313     )
314     # prefetch_factor 只有在多进程时有效
315     if num_workers > 0 and prefetch_factor is not None:
316         kwargs["prefetch_factor"] = int(prefetch_factor)
317     if persistent_workers:
318         kwargs["persistent_workers"] = True
319     return DataLoader(ds, **kwargs)
320
321
322 def get_dataloaders_hybrid(
323     # --- 新用法: 与 train.py / evaluate.py 一致 ---
324     csv_base: Optional[str] = None,
325     img_base: Optional[str] = None,
326     batch_size: int = 64,
327     num_workers: int = 4,
328     pin_memory: bool = True,
329     persistent_workers: bool = False,

```



```

14 # 与 train.py 保持一致的最小 CFG (可按需改路径)
15 CFG: Dict[str, object] = dict(
16     device="cuda" if torch.cuda.is_available() else "cpu",
17     csv_base=os.path.abspath(os.path.join(os.path.dirname(__file__), "..", "..", "data", "csv")),
18     img_base=None,
19     save_dir=os.path.abspath(os.path.join(os.path.dirname(__file__), "..", "..", "checkpoints")),
20     best_ckpt=os.path.abspath(os.path.join(os.path.dirname(__file__), "..", "..", "checkpoints", "best_model_stage3.pth")),
21     batch_size=64,
22     num_workers=4,          # 仅占位: run_evaluation 内会强制改为 0
23     pin_memory=True,
24     persistent_workers=True, # 仅占位: run_evaluation 内会强制改为 False
25     per_class_limit=5000,
26     model_variant="large",
27 )
28
29 # 评估默认选项 (可被 run_evaluation 的 eval_overrides 覆盖)
30 EVAL_DEFAULT: Dict[str, object] = dict(
31     split="both", # "val" / "test" / "both"
32     tta=True,
33     ckpt=None,    # None → 使用 CFG["best_ckpt"]
34 )
35
36
37 # ----- 轻量评估工具 (避免循环依赖) -----
38 def _accuracy(logits: torch.Tensor, target: torch.Tensor) -> float:
39     pred = logits.argmax(1)
40     return float((pred == target).float().mean().item())
41
42
43 def _macro_f1(logits: torch.Tensor, target: torch.Tensor, num_classes: int = 7) -> float:
44     pred = logits.argmax(1)
45     f1s = []
46     for c in range(num_classes):
47         tp = ((pred == c) & (target == c)).sum().item()
48         fp = ((pred == c) & (target != c)).sum().item()
49         fn = ((pred != c) & (target == c)).sum().item()
50         p = tp / (tp + fp + 1e-8)
51         r = tp / (tp + fn + 1e-8)
52         f1s.append(2 * p * r / (p + r + 1e-8))
53     return float(sum(f1s) / len(f1s))
54
55
56 # ----- 公开入口: 供 train.py 调用, 也可直接运行 -----
57 def run_evaluation(cfg: Dict[str, object], eval_overrides: Optional[Dict[str, object]] = None) -> Dict[str, object]:
58     """
59     高级评估入口, 返回 summary 字典 (含 val/test 指标)。
60     **Windows 修复**: 评估阶段强制单进程 DataLoader [num_workers=0, persistent_workers=False]。
61     """
62     # 合并配置
63     E = dict(EVAL_DEFAULT)
64     if eval_overrides:
65         E.update(eval_overrides)
66
67     os.makedirs(str(cfg["save_dir"]), exist_ok=True)
68     log_csv_path = os.path.join(str(cfg["save_dir"]), "analysis_log.csv")
69
70     # --- 评估 DataLoader: 强制单进程, 避免 WinError 1114 / shm.dll 冲突 ---
71     train_loader, val_loader, test_loader, _ = get_dataloaders_hybrid(
72         csv_base=str(cfg["csv_base"]),
73         img_base=(None if cfg.get("img_base") in (None, "none", "") else str(cfg["img_base"])),
74         batch_size=int(cfg.get("batch_size", 64)),
75         num_workers=0,          # <<<<<< 关键: 评估用单进程
76         pin_memory=bool(cfg.get("pin_memory", True)),
77         persistent_workers=False, # <<<<<< 关键: 与 num_workers=0 搭配
78         dynamic_sampling=False,
79         per_class=int(cfg.get("per_class_limit", 5000)),
80         include_unlabeled=False,
81         unlabeled_two_views=False,
82         prefetch_factor=2,
83     )
84
85     device = str(cfg.get("device", "cpu"))
86     criterion = nn.CrossEntropyLoss(label_smoothing=0.0)
87     ckpt_path = str(E["ckpt"] or cfg["best_ckpt"])
88     do_tta = bool(E["tta"])
89     which = str(E["split"]).lower()
90
91     # 模型
92     print(f"[evaluate] loading checkpoint: {ckpt_path}", flush=True)
93     model = get_model(str(cfg.get("model_variant", "large")), num_classes=7,
94                       pretrained=False, device=device, verbose=False)

```

```

91 # basic utils
92 # =====
93 def log(msg: str) -> None:
94     print(msg, flush=True)
95
96
97 def ensure_dir(path: str) -> None:
98     Path(path).mkdir(parents=True, exist_ok=True)
99
100
101 def remove_path(path: str) -> None:
102     p = Path(path)
103     if p.is_dir():
104         shutil.rmtree(p)
105     elif p.exists():
106         p.unlink()
107
108
109 def to_abs(path: str) -> str:
110     return str(Path(path).resolve())
111
112
113 def get_device(device_name: str) -> torch.device:
114     if str(device_name).lower() == "cuda" and torch.cuda.is_available():
115         return torch.device("cuda")
116     return torch.device("cpu")
117
118
119 def interpolation_mode(name: str) -> int:
120     name = str(name).strip().lower()
121     if name == "nearest":
122         return Image.NEAREST
123     if name == "bicubic":
124         return Image.BICUBIC
125     return Image.BILINEAR
126
127
128 def tf_dtype_from_name(name: str) -> tf.dtypes.DType:
129     name = str(name).lower()

```

```

145 # =====
146 # model build / ckpt load
147 # =====
148 def build_model(cfg: Dict[str, Any]) -> nn.Module:
149     num_classes = int(cfg["num_classes"])
150
151     import_errors: List[str] = []
152
153     try:
154         from model_mbv3 import MobileNetV3Large # type: ignore
155         model = MobileNetV3Large(pretrained=False, num_classes=num_classes)
156         log(f"MobileNetV3-large initialized (pretrained=False, num_classes={num_classes})")
157         return model
158     except Exception as e:
159         import_errors.append(f"from model_mbv3 import MobileNetV3Large -> {repr(e)}")
160
161     try:
162         from model_mbv3 import mobilenet_v3_large # type: ignore
163         model = mobilenet_v3_large(num_classes=num_classes)
164         log(f"mobilenet_v3_large initialized (num_classes={num_classes})")
165         return model
166     except Exception as e:
167         import_errors.append(f"from model_mbv3 import mobilenet_v3_large -> {repr(e)}")
168
169     try:
170         from torchvision.models import mobilenet_v3_large # type: ignore
171         model = mobilenet_v3_large(num_classes=num_classes)
172         log(f"torchvision mobilenet_v3_large initialized (num_classes={num_classes})")
173         return model
174     except Exception as e:
175         import_errors.append(f"from torchvision.models import mobilenet_v3_large -> {repr(e)}")
176
177     joined = "\n".join(import_errors)
178     raise ImportError(
179         "Unable to build model. Please adjust build_model() to match your project.\n"
180         + joined
181     )
182
183

```

```

217 # =====
218 # image / csv loading
219 # =====
220 def load_csv_samples(cfg: Dict[str, Any], limit: Optional[int] = None) -> list[Dict[str, Any]]:
221     csv_path = cfg["csv_path"]
222     if not os.path.isfile(csv_path):
223         raise FileNotFoundError(f"CSV not found: {csv_path}")
224
225     df = pd.read_csv(csv_path)
226     if df.empty:
227         raise ValueError(f"CSV is empty: {csv_path}")
228
229     split_col = str(cfg.get("csv_split_col", "") or "").strip()
230     split_value = cfg.get("csv_split_value", "")
231     if split_col and split_col in df.columns and split_value != "":
232         df = df[df[split_col] == split_value]
233
234     if bool(cfg.get("csv_shuffle", False)):
235         df = df.sample(frac=1.0, random_state=int(cfg.get("csv_shuffle_seed", 42))).reset_index(drop=True)
236
237     mode = str(cfg.get("csv_mode", "image_path")).strip().lower()
238
239     if mode == "pixels":
240         pixels_col = str(cfg.get("csv_pixels_col", "pixels"))
241         label_col = str(cfg.get("csv_label_col", "emotion"))
242
243         if pixels_col not in df.columns:
244             raise KeyError(
245                 f"CSV does not contain pixels column '{pixels_col}'. Actual columns: {list(df.columns)}"
246             )
247
248         samples: list[Dict[str, Any]] = []
249         for _, row in df.iterrows():
250             item: Dict[str, Any] = {"pixels": row[pixels_col]}
251             if label_col in df.columns:
252                 item["label"] = int(row[label_col])
253             samples.append(item)
254
255     else:

```

```

43 # =====
44 # 工具函数
45 # =====
46 def seed_all(seed: int = 42):
47     import random
48     random.seed(seed)
49     np.random.seed(seed)
50     torch.manual_seed(seed)
51     torch.cuda.manual_seed_all(seed)
52
53
54 class IndexedDataset(Dataset):
55     """包装一个 Dataset, 使 __getitem__ 同时返回 (data, idx)"""
56     def __init__(self, base: Dataset):
57         self.base = base
58
59     def __len__(self):
60         return len(self.base)
61
62     def __getitem__(self, idx: int):
63         x, _ = self.base[idx]
64         return x, idx
65
66
67 def _make_loader(ds: Dataset, batch_size: int, shuffle: bool,
68                 num_workers: int = 4, pin_memory: bool = True) -> DataLoader:
69     kwargs = dict(
70         batch_size=batch_size,
71         shuffle=shuffle,
72         num_workers=num_workers,
73         pin_memory=pin_memory,
74         drop_last=False,
75     )
76     if num_workers > 0:
77         kwargs["prefetch_factor"] = 2
78         kwargs["persistent_workers"] = True
79     return DataLoader(ds, **kwargs)
80
81

```

```

82 # =====
83 # 第一阶段：伪标签生成（带 tqdm）
84 # =====
85 @torch.no_grad()
86 def generate_pseudo_first_stage(cfg: Dict[str, object]):
87     """
88     第一阶段伪标签生成：使用教师模型推理 unlabeled 数据并生成 pseudo_labeled.csv
89     """
90     device = str(cfg.get("device", "cpu"))
91     seed_all(int(cfg.get("seed", 42)))
92
93     os.makedirs(str(cfg["save_dir"]), exist_ok=True)
94
95     out_csv_path = os.path.join(cfg["save_dir"], cfg["out_csv_name"])
96     out_stats_path = os.path.join(cfg["save_dir"], cfg["out_stats_name"])
97
98     # -----
99     # 找 unlabeled.csv
100     # -----
101     unlabeled_csv = cfg.get("unlabeled_csv")
102     if unlabeled_csv in (None, "", "None"):
103         base = str(cfg["csv_base"])
104         cand = [f for f in os.listdir(base) if f.lower().endswith(".csv")]
105         u_list = [f for f in cand if "unlabeled" in f.lower()]
106         if not u_list:
107             raise FileNotFoundError("✗ 未找到 unlabeled CSV")
108         unlabeled_csv = os.path.join(base, sorted(u_list)[0])
109
110     unlabeled_csv = os.path.abspath(unlabeled_csv)
111     print(f"[Stage1] Using unlabeled CSVs: (unlabeled_csv)")
112
113     # -----
114     # DataLoader
115     # -----
116     u_set = FER2013Hybrid(
117         csv_path=unlabeled_csv,
118         img_root=(None if cfg["img_base"] in (None, "", "None") else cfg["img_base"]),
119         split="unlabeled",
120         img_size=cfg["img_size"],
121         tan_views=False,
122
123
124
125     # -----
126     # 加载 Teacher 模型
127     # -----
128     model = get_model(
129         variant="large",
130         num_classes=cfg["num_classes"],
131         pretrained=False,
132         device=device,
133         verbose=True,
134         compile_model=False,
135     )
136
137     ckpt = cfg["teacher_ckpt"]
138     print(f"[Stage1] Loading teacher checkpoint: {ckpt}")
139
140     state = torch.load(ckpt, map_location=device)
141     if isinstance(state, dict) and "state_dict" in state:
142         state = state["state_dict"]
143
144     model.load_state_dict(state)
145     model.to(device).eval()
146
147     # -----
148     # 伪标签生成
149     # -----
150     min_conf = cfg["min_conf"]
151     max_per_class = cfg["max_per_class"]
152     num_classes = cfg["num_classes"]
153
154     per_class_counts = [0] * num_classes
155     selected_indices, selected_labels, selected_confs = [], [], []
156
157     tta = cfg["tta"]
158
159     print(f"[Stage1] --- Generating pseudo labels with tqdm ---")
160
161     for xb, idx in tqdm(loader, desc="Stage1 Generating Pseudo", ncols=100):
162         xb = xb.to(device, non_blocking=True)

```

```

199 # -----
200 # 写入 CSV
201 # -----
202 import csv
203 rows = []
204 for i, y, c in zip(selected_indices, selected_labels, selected_confs):
205     s = u_set.samples[i]
206     rows.append({
207         "label": y,
208         "pixels": s.get("pixels", ""),
209         "path": s.get("path", ""),
210         "usage": "pseudo",
211         "conf": f"{c:.6f}",
212     })
213
214 with open(out_csv_path, "w", newline="", encoding="utf-8") as f:
215     writer = csv.DictWriter(f, fieldnames=["label", "pixels", "path", "usage", "conf"])
216     writer.writeheader()
217     writer.writerows(rows)
218
219 print(f"[Stage1] Saved pseudo_labeled.csv + {out_csv_path}")
220
221 # -----
222 # 保存统计信息
223 # -----
224 stats = dict(
225     total_unlabeled=len(u_set),
226     selected=len(selected_indices),
227     per_class_counts=per_class_counts,
228     min_conf=min_conf,
229     max_per_class=max_per_class,
230     unlabeled_csv=unlabeled_csv,
231     out_csv=out_csv_path,
232 )
233
234 with open(out_stats_path, "w", encoding="utf-8") as f:
235     json.dump(stats, f, indent=2, ensure_ascii=False)
236
237

```

```

1 import torch
2 import torch.nn as nn
3 from torchvision import models
4 from torchvision.models import MobileNet_V3_Small_Weights, MobileNet_V3_Large_Weights
5
6 def get_model(variant="large", num_classes=7, pretrained=True, device="cuda", verbose=True, compile_model=False):
7     variant = variant.lower()
8     if variant == "small":
9         weights = MobileNet_V3_Small_Weights.DEFAULT if pretrained else None
10        model = models.mobilenet_v3_small(weights=weights)
11    else:
12        weights = MobileNet_V3_Large_Weights.DEFAULT if pretrained else None
13        model = models.mobilenet_v3_large(weights=weights)
14
15    if hasattr(model, "classifier"):
16        last = model.classifier[-1]
17        in_features = last.in_features if isinstance(last, nn.Linear) else 1280
18        model.classifier[-1] = nn.Linear(in_features, num_classes)
19    else:
20        model = nn.Sequential(model, nn.AdaptiveAvgPool2d(1), nn.Flatten(1), nn.Linear(1280, num_classes))
21
22    model.to(device)
23    if compile_model and hasattr(torch, "compile"):
24        try:
25            model = torch.compile(model)
26        except Exception:
27            pass
28    if verbose:
29        print(f"MobileNetV3-{variant} initialized (pretrained={pretrained}, num_classes={num_classes})")
30    return model
31

```



```

69 # -----
70 # 小工具
71 # -----
72 def seed_all(seed: int = 42):
73     random.seed(seed)
74     np.random.seed(seed)
75     torch.manual_seed(seed)
76     torch.cuda.manual_seed_all(seed)
77
78
79 def cosine_warmup_lr(base_lr: float, floor: float,
80                     warmup_epochs: int, total_epochs: int, epoch: int) -> float:
81     """线性 warmup + cosine 衰减"""
82     if epoch < warmup_epochs:
83         return base_lr * float(epoch + 1) / max(1, warmup_epochs)
84     progress = (epoch - warmup_epochs) / max(1, total_epochs - warmup_epochs)
85     return floor + (base_lr - floor) * 0.5 * (1 + math.cos(math.pi * progress))
86
87
88 def accuracy(logits: torch.Tensor, target: torch.Tensor) -> float:
89     return float((logits.argmax(1) == target).float().mean().item())
90
91
92 def macro_f1(logits: torch.Tensor, target: torch.Tensor, C: int = 7) -> float:
93     pred = logits.argmax(1)
94     fis = []
95     for c in range(C):
96         tp = ((pred == c) & (target == c)).sum().item()
97         fp = ((pred == c) & (target != c)).sum().item()
98         fn = ((pred != c) & (target == c)).sum().item()
99         p = tp / (tp + fp + 1e-8)
100         r = tp / (tp + fn + 1e-8)
101         fis.append(2 * p * r / (p + r + 1e-8))
102     return float(np.mean(fis))
103
104
105 def _iter_labels_from_dataset(ds) -> Iterable[int]:
106     """
107     与你原 train.py 中逻辑一致：稳健获取所有标签，用于统计类别数量
108     """

```

```

139 # -----
140 # 一个 epoch 的训练
141 # -----
142 def train_one_epoch(model, optimizer, loader, device, epoch, CFG, criterion_sup, scaler=None):
143     model.train()
144     total_loss = total_acc = total_f1 = 0.0
145     total_n = 0
146
147     use_mixup = bool(CFG.get("use_mixup", False))
148     alpha = float(CFG.get("mixup_alpha", 0.2))
149
150     for xb, yb in loader:
151         xb = xb.to(device, non_blocking=True)
152         yb = yb.to(device, non_blocking=True)
153
154         optimizer.zero_grad(set_to_none=True)
155
156         if use_mixup:
157             lam = np.random.beta(alpha, alpha)
158             idx = torch.randperm(xb.size(0), device=device)
159             mixed = lam * xb + (1 - lam) * xb[idx]
160
161             with torch.amp.autocast(AMP_DEVICE, enabled=bool(CFG.get("use_amp", False))):
162                 logits = model(mixed)
163                 loss = lam * criterion_sup(logits, yb) + (1 - lam) * criterion_sup(logits, yb[idx])
164         else:
165             with torch.amp.autocast(AMP_DEVICE, enabled=bool(CFG.get("use_amp", False))):
166                 logits = model(xb)
167                 loss = criterion_sup(logits, yb)
168
169         if scaler is not None:
170             scaler.scale(loss).backward()
171             if bool(CFG.get("grad_clip", False)):
172                 scaler.unscale_(optimizer)
173                 torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=float(CFG.get("max_norm", 1.0)))
174             scaler.step(optimizer)
175             scaler.update()
176         else:
177             loss.backward()

```

```

192 # -----
193 # 验证
194 # -----
195 @torch.no_grad()
196 def evaluate(model, loader, device, CFG):
197     model.eval()
198     total_loss = total_acc = total_f1 = 0.0
199     total_n = 0
200
201     for xb, yb in loader:
202         xb = xb.to(device, non_blocking=True)
203         yb = yb.to(device, non_blocking=True)
204
205         # 这里不再使用 autograd, 直接前向即可
206         logits = model(xb)
207         loss = F.cross_entropy(logits, yb)
208
209         bs = yb.size(0)
210         total_loss += float(loss.item()) * bs
211         total_acc += accuracy(logits, yb) * bs
212         total_f1 += macro_f1(logits, yb) * bs
213         total_n += bs
214
215     return total_loss / max(1, total_n), total_acc / max(1, total_n), total_f1 / max(1, total_n)
216
217 # -----
218 # 主流程
219 # -----
220 def main():
221     os.makedirs(str(CFG["save_dir"]), exist_ok=True)
222     for k in ("log_csv", "bestckpt"):
223         d = os.path.dirname(str(CFG[k]))
224         if d:
225             os.makedirs(d, exist_ok=True)
226
227     seed_all(int(CFG["seed"]))
228     device = str(CFG["device"])
229     print(f"Device: {device}", flush=True)
230

```

```

147 # -----
148 # 伪标签置信度读取
149 # -----
150 def read_pseudo_confs(
151     csv_path: str,
152     usage_keep: Tuple[str, ...] = ("pseudo", "unlabeled", "0"),
153     conf_min: float = 0.0
154 ) -> List[float]:
155     """
156     从伪标签CSV读取置信度列表, 按Usage过滤
157
158     返回:
159     ... confs: 与CSV行数一一对应的置信度列表 (非目标Usage为0.0会被过滤)
160     """
161     confs: List[float] = []
162     with open(csv_path, "r", encoding="utf-8") as f:
163         reader = csv.DictReader(f)
164         fieldnames = [fn.lower() for fn in (reader.fieldnames or [])]
165         has_usage = "usage" in fieldnames
166
167         for row in reader:
168             usage = (row.get("Usage") or row.get("usage") or "").lower()
169             if has_usage and (usage not in usage_keep):
170                 # 非目标Usage, 标记为-1 (后续过滤)
171                 confs.append(-1.0)
172                 continue
173
174             c = row.get("conf", row.get("Conf", None))
175             try:
176                 c = float(c) if c is not None else 1.0
177             except Exception:
178                 c = 1.0
179
180             if c < float(conf_min):
181                 confs.append(-1.0) # 低于阈值, 标记为无效
182             else:
183                 confs.append(float(c))
184
185     return confs
186

```

```

188 # =====
189 # 带权重的数据集包装 (Stage2核心: 区分真实/伪标签权重)
190 # =====
191 class WeightedDataset(Dataset):
192     """
193     为每个样本附加权重, 支持真实样本与伪标签样本的差异化处理
194
195     权重设计:
196     - 真实样本: weight = 1.0 (基准)
197     - 伪标签样本: weight = (conf^power) * scale * ramp(t)
198     """
199     def __init__(
200         self,
201         base: Dataset,
202         weights: Optional[List[float]] = None,
203         is_pseudo_flags: Optional[List[bool]] = None,
204         default_w: float = 1.0
205     ):
206         self.base = base
207         self.default_w = float(default_w)
208         self.weights = weights
209         self.is_pseudo_flags = is_pseudo_flags # 标记哪些是伪标签样本
210
211         # 验证长度一致性
212         if self.weights is not None:
213             assert len(self.weights) == len(base), \
214                 f"weights长度({len(self.weights)})与数据集({len(base)})不匹配"
215         if self.is_pseudo_flags is not None:
216             assert len(self.is_pseudo_flags) == len(base), \
217                 f"is_pseudo_flags长度({len(self.is_pseudo_flags)})与数据集不匹配"
218
219     def __len__(self):
220         return len(self.base)
221
222     def __getitem__(self, idx: int):
223         x, y = self.base[idx]
224         w = self.default_w if (self.weights is None) else self.weights[idx]
225         is_pseudo = False if (self.is_pseudo_flags is None) else self.is_pseudo_flags[idx]
226         # 返回5元组: (图像, 标签, 权重, 是否伪标签, 索引)
227
228
229 # =====
230 # 类别均衡权重计算 (CB loss)
231 # =====
232 def compute_cb_weights(
233     labels: List[int],
234     num_classes: int,
235     beta: float,
236     device: str
237 ) -> torch.Tensor:
238     """
239     计算Class-Balanced权重:  $w_c \propto (1-\beta)/(1-\beta^{n_c})$ 
240     """
241     counts = np.bincount(np.array(labels, dtype=np.int64), minlength=num_classes).astype(np.float32)
242     # 防止除零
243     counts = np.maximum(counts, 1.0)
244
245     # effective number
246     eff_num = 1.0 - np.power(beta, counts)
247     cb = (1.0 - beta) / np.maximum(eff_num, 1e-8)
248     # 归一化
249     cb = cb / cb.mean()
250
251     return torch.tensor(cb, dtype=torch.float32, device=device)
252
253
254 def extract_labels(ds: Dataset) -> List[int]:
255     """从数据集提取所有标签 (用于CB权重统计)"""
256     labs: List[int] = []
257     for i in range(len(ds)):
258         y = ds[i]
259         y = int(y)
260         if y >= 0:
261             labs.append(y)
262     return labs
263
264
265 # =====
266 # 模型加载
267 # =====

```

```

432
433 # =====
434 # 主流程
435 # =====
436 def main():
437     # ----- 初始化 -----
438     os.makedirs(str(CFG["save_dir"]), exist_ok=True)
439     for k in ("log_csv", "bestckpt"):
440         d = os.path.dirname(str(CFG[k]))
441         if d:
442             os.makedirs(d, exist_ok=True)
443
444     seed_all(int(CFG["seed"]))
445     device = str(CFG["device"])
446     print(f"[Stage2] 设备: {device}")
447
448     # ----- 数据集构造 -----
449     img_root = None if CFG["img_base"] in (None, "", "None") else str(CFG["img_base"])
450
451     train_csv = os.path.abspath(str(CFG["train_csv"]))
452     pseudo_csv = os.path.abspath(str(CFG["pseudo_csv"]))
453     val_csv = os.path.abspath(str(CFG["val_csv"]))
454     test_csv = os.path.abspath(str(CFG["test_csv"]))
455
456     # 真实标注数据
457     ds_real = FER2013Hybrid(
458         train_csv, img_root, "train",
459         img_size=int(IMG_SIZE),
460         two_views=False,
461         include_label=True
462     )
463
464     # 伪标签数据: 先用"unlabeled"过fit usage=pseudo, 再改为train用训练增强
465     ds_pseudo = FER2013Hybrid(
466         pseudo_csv, img_root, "unlabeled",
467         img_size=int(IMG_SIZE),
468         two_views=False,
469         include_label=True
470     )
471     ds_pseudo.split = "train" # 关键: 转换为train模式以应用训练增强

```

```

91 # =====
92 # 工具函数
93 # =====
94 def seed_all(seed: int = 42):
95     """设置全局随机种子"""
96     random.seed(seed)
97     np.random.seed(seed)
98     torch.manual_seed(seed)
99     torch.cuda.manual_seed_all(seed)
100
101
102 def cosine_warmup_lr(base_lr: float, floor: float, warmup_epochs: int, total_epochs: int, epoch: int) -> float:
103     """线性 warmup + cosine 衰减"""
104     if epoch < warmup_epochs:
105         return base_lr * float(epoch + 1) / max(1, warmup_epochs)
106     progress = (epoch - warmup_epochs) / max(1, total_epochs - warmup_epochs)
107     return floor + (base_lr - floor) * 0.5 * (1 + math.cos(math.pi * progress))
108
109
110 def accuracy(logits: torch.Tensor, target: torch.Tensor) -> float:
111     """计算批次准确率"""
112     return float((logits.argmax(1) == target).float().mean().item())
113
114
115 def macro_f1(logits: torch.Tensor, target: torch.Tensor, num_classes: int = 7) -> float:
116     """计算宏平均 F1"""
117     pred = logits.argmax(1)
118     f1s = []
119     for c in range(num_classes):
120         tp = ((pred == c) & (target == c)).sum().item()
121         fp = ((pred == c) & (target != c)).sum().item()
122         fn = ((pred != c) & (target == c)).sum().item()
123         p = tp / (tp + fp + 1e-8)
124         r = tp / (tp + fn + 1e-8)
125         f1s.append(2 * p * r / (p + r + 1e-8))
126     return float(np.mean(f1s))
127
128
129 def make_loader(ds: Dataset, batch_size: int, shuffle: bool, cfg: Dict) -> DataLoader:

```

须知：

- 报告编号系送检论文检测报告在本系统中的唯一编号。



- 本报告结合AI大模型自动生成，结果仅供参考。
- 报告支持中、英文内容检测。



微信公众号

唯一官网: <https://vpcs.fanyu.com> | 客服邮箱: vpcs@fanyu.com | 客服热线: 400-607-5550 | 客服QQ: 4006075550

维普论文检测